

```

package com.mov.android

import android.os.Bundle
import android.widget.Button
import android.widget.EditText
import android.widget.LinearLayout
import android.widget.ScrollView
import android.widget.TextView
import android.content.Intent
import androidx.appcompat.app.AppCompatActivity
import java.io.File
import com.hermes.dsh.agentloop.AgentOptions
import com.hermes.dsh.agentloop.ReactLoopAgent
import com.hermes.dsh.brand.SessionId
import com.mov.android.md.StreamMdMachine
import com.mov.android.md.MdTableParse
import com.hermes.dsh.llm.DeepSeekAdapter
import com.hermes.dsh.llm.MockPreparer
import com.hermes.dsh.llm.Message
import com.hermes.dsh.session.SessionStore
import com.hermes.dsh.tools.MockToolScheduler
import kotlinx.coroutines.CoroutineScope
import kotlinx.coroutines.Dispatchers
import kotlinx.coroutines.cancel
import kotlinx.coroutines.launch
import kotlinx.coroutines.withContext

/**
 * dsh-kotlin Android 入口：简单对话 UI。
 * 输入问题 → DeepSeek 云端 → 显示回答 + journal 事件日志。
 */
class MainActivity : AppCompatActivity() {

    companion object {
        /**
         * Vue WebView 试点页的工具面入口（PagesBridge 按白名单前缀过滤后查这里）。
         * 由 onCreate 赋值，lambda 运行时读当前 mcpHandlers（热挂 / 摘除后仍然有效）。
         */
        var pageToolProvider: (String) -> ((Map<String, Any?>) -> Any?)? = { null }

        // P2a 商业后端（A2A relay）基地址
        const val BIZ_BASE_URL = "https://mow.kim"

        // P2b 微信支付 partner-server（特约商户进件）；试点期 token 内置（生产应下发 / 配置
    )

        const val PARTNER_BASE_URL = "https://mow.kim/partner"
        const val PARTNER_AUTH_TOKEN = "rTEEjpB1YWnXbtEpwaFXosAdwOQ77bhozHyQC_MFXSI"
    }
}

```

```

    /** 深浅模式 (false = 浅色默认——代码块 / 表格 / 高亮随模式)。 */
    /** 房间存储 (M1——meta 缓存 / 原子写 / title 派生)。 */
    private var roomStore: com.hermes.dsh.session.RoomStore? = null
    /** 铁律 1: 权限 / 模式类工具仅 UI——不进 agent 工具面, 也不注册到 MCP 服务器 (防远程自我审批)。
        * vault.peek / vault.peekPhoto 同在名单: 页面本地查看明文 / 缩略图免审批 (用户看自己数据)
, AI 明文出口只有 vault.get (审批门控)。 */
    private val uiOnlyMcpTools = setOf(
        "permission.set_mode", "permission.approve", "permission.deny", "presentation.set_mode",
        "vault.peek", "vault.peekPhoto", "vault.credPeek",
    )
    /** 渲染引擎: markstream (WebView) 为主; 渲染页加载失败自动降级原生 (不白屏)。 */
    private var markstreamMode = true
    /** markstream 渲染页加载失败置 true (onReceivedError 主帧错误) → 后续回复走原生降级。 */
    private var markstreamBroken = false
    /** markstream 渲染桥 (AI 工具面 MCP 组件用——md.renderFile 面板)。 */
    private val msMd = com.hermes.mov.tools.MarkstreamMcpBridge()
    /** 所有已创建的房间页 WebView (含缓存房间里的) ——onDestroy / LRU 驱逐统一销毁, 防 native 泄漏。 */
    private val liveMarkstreams = java.util.Collections.synchronizedList(mutableListOf<MarkstreamView>())
    /** 外部 MCP 服务器配置存储 (市场与启动发现共用同一实例)。 */
    private var mcpExtStore: com.hermes.mov.mcp.McpServerStore? = null
    /** 房间加载遮罩 (进入房间渲染好前显示)。 */
    private var roomLoadingView: android.widget.FrameLayout? = null
    /** TTS 引擎 (tts.speak 已剔除但实例仍绑定引擎——onDestroy 需 shutdown 释放)。 */
/
    private var tts: TtsProvider? = null
    private var isDark = false
    private var rootView: android.widget.LinearLayout? = null
    /** 聊天滚动容器 (进入 / 切换房间后滚到最新消息用)。 */
    private var chatScroll: android.widget.ScrollView? = null
    /** 当前房间的聊天容器 (logView + mdView 锚点 + 消息 view)。 */
    private var mdContainer: android.widget.LinearLayout? = null
    /**
        * 房间聊天视图缓存: 切房间 detach 不销毁——渲染一次终身有效 (dsh 不可变消息 + memo 的
        * Android 等价物), 二次进房 attach 即显、零渲染。LRU 最多 3 间防内存膨胀。
        */
    private val roomViewCache = object : LinkedHashMap<String, android.widget.LinearLayout>(4, 0.75f, true) {
        override fun removeEldestEntry(eldest: MutableMap.MutableEntry<String, android.widget.LinearLayout>?): Boolean {
            if (size <= 3) return false
            eldest?.value?.let { destroyWebViewsIn(it) } // 驱逐时销毁内部 WebView (防渲染进程堆积)
            return true
        }
    }

```

```

    }
    private lateinit var logView: TextView
    /** 调试日志区显隐（默认隐藏，长按房间名切换）。 */
    private var logVisible = false
    /** md 渲染区（AI 消息排版——标题 / 列表 / 代码 Markwon + 表格 TableLayout 自动对齐）。 */
    private lateinit var mdView: TextView
    private var markwon: io.noties.markwon.Markwon? = null
    private var cmParser: org.commonmark.parser.Parser? = null
    /** E3 可视化：工具面面板（当前模式下 agent 可见工具列表，切换模式实时刷新）。 */
    private lateinit var input: EditText
    private lateinit var keyInput: EditText
    private lateinit var store: SessionStore
    private var persistence: com.hermes.dsh.session.persistence.PersistenceCoordinator<*>? = null
    /** F2: FTS5 会话搜索索引。 */
    private var fts5: com.hermes.dsh.sessionquery.Fts5QueryEngine? = null
    /** F4: 凭据加密存储（Keystore AES-GCM）。 */
    private lateinit var credentials: CredentialStore
    /** D4 审批服务（审计 + 弹窗回答者）。 */
    private var approvalService: com.hermes.dsh.tools.ApprovalService? = null
    /** App 前台可见性（审批分流：可见弹窗 / 后台通知栏）。 */
    @Volatile
    private var isAppVisible = false
    /** 测试开关：长按窗口按钮 = 强制审批走通知栏（前台也走，便于验证）。 */
    @Volatile
    private var forceNotification = false
    private var session: com.hermes.dsh.session.Session? = null
    /** M2: 当前 live session 的 store release 句柄（切换房间时先退场落盘再进新房间）。 */
    /
    private var sessionRelease: (() -> Unit)? = null
    /** 流式 chunk 转发（send 设置 → runChat 里 agent 创建后挂接——agent 在 runChat 内创建，send 时还是 null）。 */
    private var streamChunkSink: ((com.hermes.dsh.llm.StreamChunk) -> Unit)? = null
    /** 侧边栏抽屉（DrawerLayout） / 面板 / 面板内 WebView（SidebarNav 页） / 面板宽（px）。 */
    private var drawerLayout: androidx.drawerlayout.widget.DrawerLayout? = null
    private var drawerPanel: android.widget.LinearLayout? = null
    private var sidebarWebView: android.webkit.WebView? = null
    /** v3 顶栏：当前房间名（居中可点，开抽屉切换）。 */
    private lateinit var roomTitleView: TextView
    /** 拍照 OCR：权限 / 拍照 launcher 与状态（拒绝后本次会话不再打扰）。 */
    private lateinit var cameraPermissionLauncher: androidx.activity.result.ActivityResultLauncher<String>
    private lateinit var takePictureLauncher: androidx.activity.result.ActivityResultLauncher<android.net.Uri>

```

```

private var pendingPhotoUri: android.net.Uri? = null
private var pendingCaptureFile: java.io.File? = null
private var cameraDeniedThisSession = false
/** 拍照用途:true=附件上传大模型(相机按钮) / false=本地 OCR 填输入框(拍照 OCR chip)。
 */
private var cameraForAttach = true
/** 待发送的图片附件(拍照提问页确认后挂到 send();发送成功清空)。 */
private var pendingPhotoFile: java.io.File? = null
@Volatile
private var ocrBusy = false
private var dockHintView: TextView? = null
private var roomListContainer: android.widget.LinearLayout? = null
private var drawerWidthPx = 0
private var agent: ReactLoopAgent? = null
/** P0:goal 目标域(set/complete 落 goal/change 事件;活跃目标注入 systemPrompt) */
private val goalDomain = com.hermes.dsh.goal.GoalDomain()
/** P0:会话引用注入(session.reference —— 每次调用从 fts5 现构(避免 lazy 单次求值锁死竞态)) */
private fun sessionReferenceResolver(): com.hermes.dsh.context.SessionReferenceResolver? =
    fts5?.let { com.hermes.dsh.context.SessionReferenceResolver(it) }
private var agentToolScheduler: com.hermes.dsh.tools.McpToolScheduler? = null
private var agentToolSchemas: List<com.hermes.dsh.llm.ToolSchema> = emptyList()
/** 市场能力仓储(builtin 安装状态查询用;onCreate 早期创建)。 */
private var marketCapability: com.hermes.mov.mcp.McpMarket? = null
/** 浏览器 AI 工具 server 侧实现(market「浏览器自动化」安装后经 BrowserMcpTools 代理挂载;名字 = server 内名)。 */
private val browserHandlers = LinkedHashMap<String, (Map<String, Any?>) -> Any?>()
private var mcpServer: com.hermes.mov.mcp.McpServer? = null
private var permissionGuard: com.hermes.dsh.tools.PermissionGuard? = null
// MCP 工具处理器(成员:rebuildAgentTools 按模式过滤需要访问)
private var mcpHandlers = LinkedHashMap<String, (Map<String, Any?>) -> Any?>()
private lateinit var modeBtn: Button
/** D3:模型切换按钮。 */
private lateinit var modelBtn: Button
// E3:工具面呈现模式(both 全量 / code 只留执行器 / native 去执行器)
private var presentationMode = com.hermes.dsh.tools.ToolPresentationMode.BOTH
/** C2 查验:上下文窗口(1M 生产 / 128K / 5K 测试),切换后真实对话可触发压缩。实测 DeepSeek 支持 ≥900K。 */
private var contextWindow = 1000000

```

```

    /** 公共目录存储 (MediaStore.Downloads; 笔记/spill/文件事实源 = /sdcard/Download/MOV/)。 */
    private lateinit var movStorage: MovStorage
    private lateinit var bizStore: com.hermes.mov.biz.BizStore
    private lateinit var onboardDraft: com.hermes.mov.biz.OnboardDraft

    /** 本地个人信息库 "我的信息" (filesDir/vault/, Keystore AES 加密; 明文读取一律走 vault.get 审批门控)。 */
    private lateinit var infoVault: com.hermes.mov.biz.InfoVault

    /** 剧本解释器 (MERCHANT_ONBOARDING_PLAN_S2; 步骤状态落盘 filesDir/workflows/)。 */
    private lateinit var workflowRunner: com.hermes.mov.workflow.WorkflowRunner

    /** P1-3: 发送中标志 (防连点并发)。 */
    @Volatile
    private var sending = false
    // E3: code 模式只保留执行器工具 (代码运行场景)
    private val codeTools = setOf("shell.exec")

    /** 工具真实参数 schema (E3: agent 工具面必须给 AI 参数定义, 否则 AI 盲调——参数永远为空)。 */
    private val toolParamSchemas: Map<String, Map<String, Any?>> =
mapOf(
    "mov.journalTail" to paramSchema(listOf(
        "roomId" to "string", "afterSeq" to "integer", "count"
to "integer",
    )),
    "volume.set" to paramSchema(listOf("level" to "integer")),
    "notification.post" to paramSchema(listOf("title" to "string", "text" to "string")),
    "xiaomi.assist" to paramSchema(listOf("utterance" to "string")),
    "note.create" to paramSchema(listOf("title" to "string", "content" to "string")),
    "text2image" to paramSchema(listOf("text" to "string", "filename" to "string")),
    "md.render" to paramSchema(listOf("md" to "string")),
    "md.renderFile" to paramSchema(listOf("md" to "string", "title" to "string")),
    "image.info" to paramSchema(listOf("path" to "string")),
    "image.ocr" to paramSchema(listOf("path" to "string")),
    "http.download" to paramSchema(listOf("url" to "string", "filename" to "string")),
    "pdf.read" to paramSchema(listOf("path" to "string")),
    "spill.list" to paramSchema(emptyList()),
    "model.list" to paramSchema(emptyList()),
    "error.classify" to paramSchema(listOf("text" to "string")),
    "session.search" to paramSchema(listOf("query" to "string", "limit" to "integer")),

```

```

// P0 (dsh 对齐) :会话引用注入 + goal 目标三件套
"session.reference" to paramSchema(listOf("ids" to "array")
),
    "goal.set" to paramSchema(listOf("id" to "string", "content
" to "string")),
    "goal.complete" to paramSchema(listOf("id" to "string")),
    "goal.status" to paramSchema(emptyList()),
    "error.report" to paramSchema(listOf("code" to "string", "m
essage" to "string")),
    "model.use" to paramSchema(listOf("id" to "string")),
    "spill.clean" to paramSchema(listOf("name" to "string")),
    "memory.save" to paramSchema(listOf("content" to "string",
"kind" to "string")),
    "memory.load" to paramSchema(listOf("query" to "string")),
    "causal.record" to paramSchema(listOf(
        "subject" to "string", "predicate" to "string", "object
" to "string", "t" to "string",
    )),
    "causal.link" to paramSchema(listOf("a" to "string", "b" to
"string", "delta" to "number", "reason" to "string")),
    "causal.query" to paramSchema(listOf("a" to "string", "b" t
o "string")),
    "http.get" to paramSchema(listOf("url" to "string")),
    "http.post" to paramSchema(listOf("url" to "string", "body"
to "string")),
    "shell.exec" to paramSchema(listOf("command" to "string")),
    "file.read" to paramSchema(listOf("path" to "string")),
    "file.write" to paramSchema(listOf("path" to "string", "con
tent" to "string")),
    "echo" to paramSchema(listOf("text" to "string")),
    "search" to paramSchema(listOf("query" to "string")),
    "subagent.run" to paramSchema(listOf("task" to "string")),
    "agent.stop" to paramSchema(emptyList()),
    "market.refresh" to paramSchema(listOf("force" to "boolean"
)),
    "market.list" to paramSchema(listOf("cat" to "string")),
    "market.install" to paramSchema(listOf("id" to "string")),
    "market.uninstall" to paramSchema(listOf("id" to "string"))
,
    "market.enable" to paramSchema(listOf("id" to "string")),
    "market.disable" to paramSchema(listOf("id" to "string")),
    "market.status" to paramSchema(listOf("id" to "string")),
    "ui.prefillInput" to paramSchema(listOf("text" to "string")
),
    "mov.subagentList" to paramSchema(listOf("roomId" to "strin
g")),
    "biz.onboardStart" to paramSchema(emptyList()),
    "biz.onboardSet" to paramSchema(listOf("key" to "string", "
value" to "string")),
    "biz.onboardPhoto" to paramSchema(listOf("kind" to "string"

```

```

, "path" to "string")),
    "biz.onboardScan" to paramSchema(listOf("kind" to "string",
"path" to "string")),
    "biz.onboardSubmit" to paramSchema(emptyList()),
    "biz.onboardStatus" to paramSchema(emptyList()),
    "vault.list" to paramSchema(emptyList()),
    "vault.set" to paramSchema(listOf("key" to "string", "value
" to "string")),
    "vault.delete" to paramSchema(listOf("key" to "string")),
    "vault.setPhoto" to paramSchema(listOf("kind" to "string",
"path" to "string")),
    "vault.scanPhoto" to paramSchema(listOf("kind" to "string",
"path" to "string")),
    "vault.get" to paramSchema(listOf("keys" to "array")),
    "vault.credSet" to paramSchema(listOf("platform" to "string
", "account" to "string", "password" to "string")),
    "vault.credDelete" to paramSchema(listOf("platform" to "str
ing")),
    // AI 可操作浏览器 (AGENT_BROWSER_PLAN 第 5 节; debug.openUrl 已收编为 b
rowser.open)
    // 剧本解释器 (MERCHANT_ONBOARDING_PLAN S2 最小闭环)
    "workflow.run" to paramSchema(listOf("id" to "string")),
    "workflow.resume" to paramSchema(listOf("id" to "string")),
    "workflow.status" to paramSchema(listOf("id" to "string"),
emptyList()),
    // MCP 市场审核口 (MARKET_REVIEW_PLAN; token 工具页面私有走 localHandler
s, 不进 MCP 面)
    "marketAdmin.pending" to paramSchema(emptyList()),
    "marketAdmin.approve" to paramSchema(listOf("id" to "string
")),
    "marketAdmin.reject" to paramSchema(listOf("id" to "string"
, "reason" to "string")),
    "ui.openMarketReview" to paramSchema(emptyList()),
)

/** 构造 JSON Schema (type: object + properties + required; require
d 默认全属性)。 */
private fun paramSchema(props: List<Pair<String, String>>): Map
<String, Any?> =
    paramSchema(props, props.map { it.first })

/** 可选参数版: required 只列调用必须的参数 (如 browser.snapshot 的 maxChars 可选
)。 */
private fun paramSchema(props: List<Pair<String, String>>, requ
ired: List<String>): Map<String, Any?> {
    val properties = props.associate { (k, t) -> k to mapOf("ty
pe" to t, "description" to k) }
    return mapOf(
        "type" to "object",
        "properties" to properties,

```

```

        "required" to required,
    )
}

private val scope = CoroutineScope(Dispatchers.Main)

override fun onCreate(savedInstanceState: Bundle?) {
    super.onCreate(savedInstanceState)
    ApplicationHolder.context = applicationContext // LightOcr (
ML Kit) 静态入口
    // WebView 内容调试 (debug 包): 必须在进程首个 WebView 创建前调用, 否则被忽略
    if (applicationInfo.flags and android.content.pm.ApplicationInfo.FLAG_DEBUGGABLE != 0) {
        android.webkit.WebView.setWebContentsDebuggingEnabled(true)
    }
    // 隐藏顶部 ActionBar (删掉标题栏)
    supportActionBar?.hide()

    // 拍照 OCR: 权限 + 拍照 launcher (官方 ActivityResult API; 须在 onStart 前注册)
    cameraPermissionLauncher = registerForActivityResult(
        androidx.activity.result.contract.ActivityResultContracts.RequestPermission(),
    ) { granted ->
        if (granted) {
            launchCamera()
        } else {
            cameraDeniedThisSession = true // 本次会话不再打扰
            android.widget.Toast.makeText(this, "未授权相机权限, 无法拍照识别", android.widget.Toast.LENGTH_SHORT).show()
        }
    }

    takePictureLauncher = registerForActivityResult(
        androidx.activity.result.contract.ActivityResultContracts.TakePicture(),
    ) { success ->
        val uri = pendingPhotoUri
        pendingPhotoUri = null
        // 用户取消拍照: 静默返回 (success=false)
        if (success && uri != null) {
            if (cameraForAttach) openPhotoAsk()
            else runOcrOnPhoto(uri)
        }
    }

    // 深浅主题统一来源 = 系统 uiMode (各 WebView 页面宿主同此判定——浅色全浅、深色全深)
    isDark = resources.configuration.uiMode and android.content.res.Configuration.UI_MODE_NIGHT_MASK ==
        android.content.res.Configuration.UI_MODE_NIGHT_YES

```

```

// Vue WebView 试网页工具面 (PagesBridge 白名单过滤后查 mcpHandlers; lambda 运行时读, 热挂后仍有效)
pageToolProvider = { tool -> mcpHandlers[tool] }

// 公共目录存储: MediaStore.Downloads (零权限、零弹窗) —— 迁移旧私有 spill
movStorage = MovStorage(this)
// P2a 商业后端 (mow.kim): 设备凭据持久化 (device_id 首后生成, token 注册后落盘)
bizStore = com.hermes.mov.biz.BizStore(this)
// P2b 商户入驻草稿 (filesDir/onboard_draft.json; 对话式进件状态机)
onboardDraft = com.hermes.mov.biz.OnboardDraft(java.io.File(
(filesDir, "onboard_draft.json"))
// 本地个人信息库 (Keystore AES/GCM 文件级加密; JVM 单测走注入假加密的 InfoVault
本体)
infoVault = com.hermes.mov.biz.InfoVault(java.io.File(files
Dir, "vault"), com.mov.android.VaultKeystoreCrypto())
workflowRunner = com.hermes.mov.workflow.WorkflowRunner(
java.io.File(filesDir, "workflows"),
opener = { u ->
    runOnUiThread {
        closeRoomDrawer()
        WebPageSheet.showAgent(this, u)
    }
},
// S3 D3: from=vault.* 明文读取——只用 InfoVault 解密直填, 进出都不经 A
I 上下文
vaultPlain = { fromKeys ->
    val out = LinkedHashMap<String, String>()
    for (fk in fromKeys) {
        val spec = com.hermes.mov.workflow.WorkflowEngi
ne.parseVaultFrom(fk) ?: continue
        when (spec) {
            is com.hermes.mov.workflow.VaultSpec.Cred -
>
                infoVault.credPlain(spec.platform)?.get
(spec.field)?.let { out[fk] = it }
            is com.hermes.mov.workflow.VaultSpec.Info -
>
                infoVault.getPlain(listOf(spec.key))[sp
ec.key]?.let { out[fk] = it }
        }
    }
    out
},
// 明文读取门控: 审批文案只列字段中文名称 (vaultSpecLabel), 绝不含值
approveVaultRead = { fromKeys ->
    val appr = approvalService
    if (appr == null) {
        false
    } else {
        val labels = fromKeys.mapNotNull { fk ->

```

```

        com.hermes.mov.workflow.WorkflowEngine.parseVaultFrom(fk)

        ?.let { com.hermes.mov.workflow.WorkflowEngine.vaultSpecLabel(it) }

        ?: fk

    }

    appr.request(
        "workflow.run",
        "从信息库读取「${labels.joinToString(", ")}」用于填写当前
        表单（明文只直填页面，不展示给 AI）",
        mapOf("fields" to fromKeys),
    ) == com.hermes.dsh.tools.ApprovalService.Companion.OUTCOME_ALLOWED_ONCE
    }

    },

)

Thread {
    try { movStorage.migrateFromPrivate("spill", "spill") }
catch (_: Exception) {}
}.start()

// 代码布局（MOV 约定：不用 XML）
val root = LinearLayout(this).apply {
    orientation = LinearLayout.VERTICAL
    // 状态栏适配（内容不被系统顶栏盖住）
    val sbRes = resources
    val sbId = sbRes.getIdentifier("status_bar_height", "dimen", "android")
    val sbH = if (sbId > 0) sbRes.getDimensionPixelSize(sbId) else 0
    setPadding(32, 32 + sbH, 32, 32)
}
rootView = root

val scroll = ScrollView(this)
logView = TextView(this).apply {
    textSize = 14f
    setTextIsSelectable(true) // 长按可选中复制
    visibility = android.view.View.GONE // 调试日志区默认隐藏（长按房间名
切换)
}

// ScrollView 只允许一个直接子 View：当前房间的聊天容器（logView + mdView 锚点
+ 消息 view)
scroll.removeAllViews()
chatScroll = scroll
val container = createChatContainer()
scroll.addView(container)
mdContainer = container
markwon = io.noties.markwon.Markwon.builder(this)
    .usePlugin(io.noties.markwon.ext.strikethrough.Strikethrough

```

```

roughPlugin.create()
    // 块间距：代码块 / 段落 / 引用前后留白（防色块粘连——dsh pre margin 16px 对齐）
    .usePlugin(object : io.noties.markwon.AbstractMarkwonPl
ugin() {
        override fun configureTheme(builder: io.noties.mark
won.core.MarkwonTheme.Builder) {
            builder
                .blockMargin(8.dp2px())
                // 标题层级梯度（默认对比太弱）
                .headingTextSizeMultipliers(floatArrayOf(1.
5f, 1.35f, 1.2f, 1.1f, 1f, 1f))
                // 引用块左侧竖条（accent 蓝）
                .blockQuoteWidth(3.dp2px())
                .blockQuoteColor(0xFF4F8CFF.toInt())
            }

            // 行内代码（路径 / 文件名 / 命令）：默认整行色块直角 → 圆角胶囊（贴合文字）
            // 行内代码：圆角胶囊（URL 代码由 appendAiMd 预处理转链接——Markwon 原
生 URLSpan 可点）
            override fun configureSpansFactory(builder: io.noti
es.markwon.MarkwonSpansFactory.Builder) {
                builder.setFactory(
                    org.commonmark.node.Code::class.java,
                    io.noties.markwon.SpanFactory { _, _ ->
                        MovCodeSpan(0xFF454D5E.toInt(), 0xFFE8E
AEE.toInt(), 6f)
                    },
                )
            }
        }
    })
    .build()
cmParser = org.commonmark.parser.Parser.builder()
    .extensions(listOf(org.commonmark.ext.gfm.tables.Tables
Extension.create()))
    .build()
// 顶栏（参考 MOV-UI-reference showcase topbar：历史左 / 新对话右——消息流
顶部）
val topbar = LinearLayout(this).apply {
    orientation = LinearLayout.HORIZONTAL
    gravity = android.view.Gravity.CENTER_VERTICAL
    layoutParams = LinearLayout.LayoutParams(
        LinearLayout.LayoutParams.MATCH_PARENT,
        LinearLayout.LayoutParams.WRAP_CONTENT,
    ).apply { bottomMargin = 4 }
    // 顶栏左右外边距（平板窗口化时不贴屏幕边）
    setPadding(10.dp2px(), 0, 10.dp2px(), 0)
}
// 无边框图标按钮（透明底 + 水波纹点击反馈——不要灰底边框包围）
fun iconButton(icon: Int, desc: String, onClick: () -> Unit
): android.widget.ImageButton =

```

```

        android.widget.ImageButton(this).apply {
            setImageResource(icon)
            if (isDark) setColorFilter(0xFFE8EAE8.toInt()) // 深色下图标提浅 (图标 xml 描边色是深灰)
            val ta = obtainStyledAttributes(intArrayOf(android.R.attr.selectableItemBackgroundBorderless))
            background = ta.getDrawable(0)
            ta.recycle()
            layoutParams = LinearLayout.LayoutParams(34.dp2px(), 34.dp2px())

            contentDescription = desc
            setOnClickListener { onClick() }
        }
// 菜单: 打开侧边栏 (WORKBENCH / HISTORY / profile+设置 三段)
topbar.addView(iconButton(R.drawable.ic_menu, "菜单") { openRoomDrawer() })
// 中: 当前房间名 (居中 + 下拉箭头, 点击弹出房间浮层——不开侧边栏)
// 房间浮层: 贴顶栏下方的小面板 (新建房间行 + RoomStore 真实房间列表 + 当前 ✓)
fun showRoomPopup(anchor: android.view.View) {
    val popText = if (isDark) 0xFFE8EAE8.toInt() else 0xFF1F2329.toInt()
    val popText2 = 0xFF8A919C.toInt()
    val popPrimary = 0xFF0E7C5B.toInt()
    val listCol = LinearLayout(this).apply { orientation = LinearLayout.VERTICAL }
    val popup = android.widget.PopupWindow(
        android.widget.ScrollView(this).apply {
            addView(listCol)
        },
        320.dp2px(),
        android.view.ViewGroup.LayoutParams.WRAP_CONTENT,
        true,
    ).apply {
        elevation = 10.dp2px().toFloat()
        isOutsideTouchable = true
        setBackgroundDrawable(android.graphics.drawable.ColorDrawable(android.graphics.Color.TRANSPARENT))
    }
    listCol.background = android.graphics.drawable.GradientDrawable().apply {
        cornerRadius = 14.dp2px().toFloat()
        setColor(if (isDark) 0xFF1F2329.toInt() else 0xFFFFFFF.toInt())
        setStroke(1.dp2px(), if (isDark) 0xFF2E333B.toInt() else 0xFFE5E7EB.toInt())
    }
    listCol.setPadding(6.dp2px(), 6.dp2px(), 6.dp2px(), 6.dp2px())

    fun popRow(title: String, right: String, checked: Boolean, bold: Boolean, onClick: () -> Unit): LinearLayout =

```

```

        LinearLayout(this).apply {
            orientation = LinearLayout.HORIZONTAL
            gravity = android.view.Gravity.CENTER_VERTICAL
            setPadding(12.dp2px(), 10.dp2px(), 12.dp2px(),
10.dp2px())

            setOnClickListener { onClick() }
            addView(TextView(this@MainActivity).apply {
                text = title
                textSize = 14f
                if (bold) typeface = android.graphics.Typef
ace.DEFAULT_BOLD

                setTextColor(popText)
                maxLines = 1
                ellipsize = android.text.TextUtils.Truncate
At.END

            }, LinearLayout.LayoutParams(0, LinearLayout.La
youtParams.WRAP_CONTENT, 1f))
            if (right.isNotEmpty()) addView(TextView(this@M
ainActivity).apply {

                text = right
                textSize = 12f
                setTextColor(popText2)
            })
            if (checked) addView(TextView(this@MainActivity
).apply {

                text = " ✓"
                textSize = 15f
                setTextColor(popPrimary)
            })
        }
    // 新建房间行 (置顶)
    listCol.addView(popRow("+ 新建房间", "", false, true) {
        popup.dismiss()
        startNewChat()
    })
    val curId = session?.id?.value
    val rooms = roomStore?.listRooms()?.filter { !it.blank
}.orEmpty()
    for (meta in rooms) {
        listCol.addView(popRow(
            meta.title.ifEmpty { "新对话" },
            humanTime(meta.updatedAt),
            meta.id == curId,
            false,
        ) {
            popup.dismiss()
            switchToRoom(meta)
        })
    }
}
// 居中对齐锚点 (房间名在顶栏中间, 浮层应在其正下方居中, 而非贴左)

```

```

        val popW = 320.dp2px()
        val xOff = (anchor.width - popW) / 2
        popup.showAsDropDown(anchor, xOff, 6.dp2px())
    }
    roomTitleView = TextView(this).apply {
        text = "新对话"
        textSize = 15f
        typeface = android.graphics.Typeface.DEFAULT_BOLD
        setTextColor(if (isDark) 0xFFE8EAE.toInt() else 0xFF1F
2329.toInt())
        maxLines = 1
        ellipsize = android.text.TextUtils.TruncateAt.END
        gravity = android.view.Gravity.CENTER
        setOnClickListener { showRoomPopup(this) }
        // 长按房间名 = 调试日志区显隐开关（默认隐藏）
        setOnLongClickListener {
            logVisible = !logVisible
            logView.visibility = if (logVisible) android.view.V
iew.VISIBLE else android.view.View.GONE
            android.widget.Toast.makeText(this@MainActivity, if
(logVisible) "日志区已显示" else "日志区已隐藏", android.widget.Toast.LENGTH_
SHORT).show()

            true
        }
    }
    topbar.addView(roomTitleView, LinearLayout.LayoutParams(0,
LinearLayout.LayoutParams.WRAP_CONTENT, 1f))
    // 右:极简模式切换(占位——功能后做, 点击 toast)
    topbar.addView(iconButton(R.drawable.ic_sun, "极简模式") {
        android.widget.Toast.makeText(this, "极简模式即将上线", androi
d.widget.Toast.LENGTH_SHORT).show()
    })
    root.addView(topbar, 0) // 索引 0 = 最顶部(状态栏下方)
    root.addView(scroll, LinearLayout.LayoutParams(
        LinearLayout.LayoutParams.MATCH_PARENT, 0, 1f))

    // API Key 输入 UI 隐藏(F4 Keystore 已保存——不显示;保留引用)
    keyInput = EditText(this).apply {
        hint = "DeepSeek API Key(sk-...)"
        visibility = android.view.View.GONE
    }

    // v3 输入区 dock (对齐 DockBar.vue 三段:chips 行 / composer 圆角 24
/ hint 行)
    val dockText = if (isDark) 0xFFE8EAE.toInt() else 0xFF1F23
29.toInt()
    val dockText2 = 0xFF8A919C.toInt()
    val dockCard = if (isDark) 0xFF1F2329.toInt() else 0xFFFF1F2
F5.toInt()
    val dockBorder = if (isDark) 0xFF2E333B.toInt() else 0xFFE5

```

```

E7EB.toInt()
    val dockPrimary = 0xFF0E7C5B.toInt()
    fun capsule(border: Int): android.graphics.drawable.Gradient
Drawable =
        android.graphics.drawable.GradientDrawable().apply {
            cornerRadius = 16.dp2px().toFloat()
            setColor(dockCard)
            setStroke(1.dp2px(), border)
        }
    val dock = LinearLayout(this).apply {
        orientation = LinearLayout.VERTICAL
        // dock 左右外边距 (平板窗口化时 chips/composer 不贴屏幕边)
        setPadding(16.dp2px(), 4.dp2px(), 16.dp2px(), 6.dp2px()
    )
    }
    // 1. chips 行: 模式 chip (bottom sheet 占位) + 功能 chips (toast 占位)
    val chipsRow = LinearLayout(this).apply {
        orientation = LinearLayout.HORIZONTAL
        gravity = android.view.Gravity.CENTER_VERTICAL
    }
    fun chipIcon(icon: Int, tint: Int): android.widget.ImageVie
w =
        android.widget.ImageView(this).apply {
            setImageResource(icon)
            setColorFilter(tint)
            layoutParams = LinearLayout.LayoutParams(14.dp2px()
, 14.dp2px()).apply { marginEnd = 4.dp2px() }
        }
    fun chipLabel(label: String, color: Int): TextView = TextVi
ew(this).apply {
        text = label
        textSize = 13f
        setTextColor(color)
    }
    // A 方案: 模式 chip (快速 既有抽屉) —— 选择写 prefs 接后端 (修复: 原占位只改文案)
    var currentModeName = if (chatModePref() == com.hermes.dsh.
llm.ChatMode.DEEP) "深度思考" else "快速"
    val modeChipLabel = chipLabel(currentModeName + " ", dockP
rimary)
    // 2026-08-25: 展开按钮图标引用 (goal / 默认 (审批) 状态跟随——showModeSheet 内刷新
使用)
    var modeChipIconView: android.widget.ImageView? = null
    // 2026-08-25: 只留快速 / 深度思考 (编程 / 写作删除); 每卡右侧 goal / 审批开关
    val modeOptions = listOf(
        "快速" to "日常问答, 响应更快",
        "深度思考" to "复杂推理, 逐步分析后再回答",
    )
    // 浮层锚点 (modeChip 构建时赋值——声明顺序先于 showModeSheet)
    var modeChipAnchor: android.view.View? = null
    fun showModeSheet() {

```

```

        val anchor = modeChipAnchor ?: return
        // 贴模式 chip 上方的轻量浮层 (PopupWindow) ——不是模态弹窗：
        // 点选即关闭并更新 chip 文案；点面板外任意处消失 (outsideTouchable)
        val sheet = LinearLayout(this).apply {
            orientation = LinearLayout.VERTICAL
            background = android.graphics.drawable.GradientDraw
            able().apply {
                cornerRadius = 14.dp2px().toFloat()
                setColor(if (isDark) 0xFF1F2329.toInt() else 0x
                FFFFFFFF.toInt())
                setStroke(1.dp2px(), dockBorder)
            }
            setPadding(3.dp2px(), 3.dp2px(), 3.dp2px(), 3.dp2px
            ())
        }
        val popup = android.widget.PopupWindow(
            sheet,
            180.dp2px(),
            android.view.ViewGroup.LayoutParams.WRAP_CONTENT,
            true, // focusable——返回键可关
        ).apply {
            elevation = 10.dp2px().toFloat()
            isOutsideTouchable = true
            // 透明背景是 outside touch 消失 + 圆角阴影生效的前提
            setBackgroundDrawable(android.graphics.drawable.Col
            orDrawable(android.graphics.Color.TRANSPARENT))
        }
        // 每卡右侧 goal 图标 (闪电=goal；眼睛=默认(审批)；点击切换——全局状态，两卡同步 +
        展开按钮跟随)

        val goalIcons = mutableListOf<android.widget.ImageView>
        ()

        fun refreshGoalIcons() {
            val r = if (goalModePref()) R.drawable.ic_zap else
            R.drawable.ic_eye
            goalIcons.forEach { it.setImageResource(r) }
            modeChipIconView?.setImageResource(r)
        }
        for ((name, desc) in modeOptions) {
            val selected = name == currentModeName
            // 2026-08-25：选中卡灰色加深 (浅色 #E8EAED / 深色 #3A3F46)
            val row = LinearLayout(this).apply {
                orientation = LinearLayout.HORIZONTAL
                gravity = android.view.Gravity.CENTER_VERTICAL
                background = android.graphics.drawable.Gradient
                Drawable().apply {
                    cornerRadius = 10.dp2px().toFloat()
                    setColor(
                        if (selected) {
                            if (isDark) 0xFF3A3F46.toInt() else
                            0xFFE8EAED.toInt()

```

```

        } else android.graphics.Color.TRANSPARENT,
NT,
        )
    }
    setPadding(3.dp2px(), 5.dp2px(), 3.dp2px(), 5.dp2px())

    setOnClickListener {
        currentModeName = name
        modeChipLabel.text = name + "  "
        applyChatModePref(
            if (name == "深度思考") com.hermes.dsh.llm.
ChatMode.DEEP
                else com.hermes.dsh.llm.ChatMode.QUICK,
        )
        android.widget.Toast.makeText(this@MainActi
vity, "已切换到「" + name + "」模式", android.widget.Toast.LENGTH_SHORT).s
how()

        popup.dismiss()
    }
}

val left = LinearLayout(this).apply { orientation =
LinearLayout.VERTICAL }
left.addView(TextView(this).apply {
    text = name
    textSize = 12f
    typeface = android.graphics.Typeface.DEFAULT_BO
LD
        setTextColor(dockText)
    })
row.addView(left, LinearLayout.LayoutParams(0, Line
arLayout.LayoutParams.WRAP_CONTENT, 1f))
// 2026-08-25:图标取代 Switch 与文字 (闪电=goal / 眼睛=默认 (审批)
)

val goalIcon = android.widget.ImageView(this).apply
{
    setImageResource(if (goalModePref()) R.drawable
.ic_zap else R.drawable.ic_eye)
    alpha = if (goalModePref()) 1f else 0.55f
    setPadding(3.dp2px(), 3.dp2px(), 3.dp2px(), 3.d
p2px())

    contentDescription = "goal 模式开关 (闪电=goal, 眼睛=默认审
批) "

    setOnClickListener {
        val next = !goalModePref()
        applyGoalModePref(next)
        refreshGoalIcons()
    }
}

goalIcons.add(goalIcon)
// 图标固定右列 (两卡上下对齐;行内 weight 撑开间隙)

```

```

        row.addView(goalIcon, LinearLayout.LayoutParams(24.
dp2px(), 24.dp2px()))
        sheet.addView(row)
    }

    // 先量面板高度,再锚到 chip 正上方(yoff = -(chip 高 + 面板高 + 间距))
    sheet.measure(
        android.view.View.MeasureSpec.makeMeasureSpec(180.d
p2px(), android.view.View.MeasureSpec.EXACTLY),
        android.view.View.MeasureSpec.makeMeasureSpec(0, an
droid.view.View.MeasureSpec.UNSPECIFIED),
    )
    popup.showAsDropDown(anchor, 0, -(anchor.height + sheet
.measuredHeight + 6.dp2px()))
}
// 2026-08-25:展开按钮图标跟随 goal / 默认(审批)状态(闪电=goal / 眼睛=默认审批)
val modeChip = LinearLayout(this).apply {
    orientation = LinearLayout.HORIZONTAL
    gravity = android.view.Gravity.CENTER_VERTICAL
    background = capsule(dockPrimary)
    setPadding(10.dp2px(), 5.dp2px(), 10.dp2px(), 5.dp2px()
)

    setOnClickListener { showModeSheet() }
    val iconV = chipIcon(if (goalModePref()) R.drawable.ic_
zap else R.drawable.ic_eye, dockPrimary)
    modeChipIconView = iconV
    addView(iconV)
    addView(modeChipLabel)
}
modeChipAnchor = modeChip
fun funcChip(icon: Int, label: String, onClick: (() -> Unit
)? = null): LinearLayout = LinearLayout(this).apply {
    orientation = LinearLayout.HORIZONTAL
    gravity = android.view.Gravity.CENTER_VERTICAL
    background = capsule(dockBorder)
    setPadding(10.dp2px(), 5.dp2px(), 10.dp2px(), 5.dp2px()
)

    layoutParams = LinearLayout.LayoutParams(
        LinearLayout.LayoutParams.WRAP_CONTENT,
        LinearLayout.LayoutParams.WRAP_CONTENT,
    ).apply { marginStart = 8.dp2px() }
    setOnClickListener {
        if (onClick != null) {
            onClick()
        } else {
            android.widget.Toast.makeText(this@MainActivity
, label + ":即将上线", android.widget.Toast.LENGTH_SHORT).show()
        }
    }
    addView(chipIcon(icon, dockText2))

```

```

        addView(chipLabel(label, dockText2))
    }
    chipsRow.addView(modeChip)
    chipsRow.addView(funcChip(R.drawable.ic_camera, "拍照 OCR") {
onCameraClick(false) })
    chipsRow.addView(funcChip(R.drawable.ic_orders, "总结文件"))
    dock.addView(chipsRow, LinearLayout.LayoutParams(
        LinearLayout.LayoutParams.MATCH_PARENT,
        LinearLayout.LayoutParams.WRAP_CONTENT,
    ).apply { bottomMargin = 8.dp2px() })
    // 2. composer:圆角 24 容器(相机占位 / 多行自增高输入 / 语音↔发送互换)
    val composer = LinearLayout(this).apply {
        orientation = LinearLayout.HORIZONTAL
        // 关闭基线对齐:否则水平 LinearLayout 默认按 EditText 文本基线排布子视图,
        // 相机/语音/发送按钮会被拉到文字基线行("飘进输入框"),而非容器垂直居中
        isBaselineAligned = false
        gravity = android.view.Gravity.CENTER_VERTICAL
        background = android.graphics.drawable.GradientDrawable
    ).apply {
        cornerRadius = 24.dp2px().toFloat()
        setColor(dockCard)
        setStroke(1.dp2px(), dockBorder)
    }
    setPadding(6.dp2px(), 4.dp2px(), 6.dp2px(), 4.dp2px())
}
fun roundIconBtn(icon: Int, desc: String, solid: Boolean, on
nClick: () -> Unit): android.widget.ImageButton =
    android.widget.ImageButton(this).apply {
        setImageResource(icon)
        contentDescription = desc
        // 深色下非实心按钮图标提浅(图标 xml 描边 #4E5560 在深色卡片上发闷)
        if (!solid && isDark) setColorFilter(0xFF9AA0A6.toInt())
        background = android.graphics.drawable.GradientDraw
able().apply {
            shape = android.graphics.drawable.GradientDrawa
ble.OVAL
            setColor(if (solid) dockPrimary else android.gr
aphics.Color.TRANSPARENT)
        }
        layoutParams = LinearLayout.LayoutParams(38.dp2px()
, 38.dp2px())
        setPadding(9.dp2px(), 9.dp2px(), 9.dp2px(), 9.dp2px
())
        setOnClickListener { onClick() }
    }
    composer.addView(roundIconBtn(R.drawable.ic_camera, "相机", f
alse) {
        onCameraClick(true)
    })
}

```

```

        input = EditText(this).apply {
            hint = "给 MOV AI 发消息..."
            textSize = 15f
            setTextColor(dockText)
            setHintTextColor(dockText2)
            background = null
            maxLines = 4 // 多行自增高 (上限 4 行)
            inputType = android.text.InputType.TYPE_CLASS_TEXT or
                android.text.InputType.TYPE_TEXT_FLAG_MULTI_LINE or
                android.text.InputType.TYPE_TEXT_FLAG_CAP_SENTENCES
            setPadding(8.dp2px(), 8.dp2px(), 8.dp2px(), 8.dp2px())
        }
        composer.addView(input, LinearLayout.LayoutParams(0, LinearLayout.LayoutParams.WRAP_CONTENT, 1f))
        val micBtn = roundIconBtn(R.drawable.ic_mic, "语音输入", false)
    {
        android.widget.Toast.makeText(this, "语音输入：即将上线", android.widget.Toast.LENGTH_SHORT).show()
    }
        val sendBtn = roundIconBtn(R.drawable.ic_send, "发送", true)
    { send() }
        sendBtn.visibility = android.view.View.GONE // 空输入显示麦克风，有字
显示发送
        composer.addView(micBtn)
        composer.addView(sendBtn)
        input.addTextChangedListener(object : android.text.TextWatcher {
            override fun beforeTextChanged(s: CharSequence?, start: Int, count: Int, after: Int) {}
            override fun onTextChanged(s: CharSequence?, start: Int, before: Int, count: Int) {}
            override fun afterTextChanged(s: android.text.Editable?) {
                val hasText = !s.isNullOrEmpty()
                sendBtn.visibility = if (hasText) android.view.View.VISIBLE else android.view.View.GONE
                micBtn.visibility = if (hasText) android.view.View.GONE else android.view.View.VISIBLE
            }
        })
        dock.addView(composer)
        // 3. hint 行 (OCR 识别中显示"识别中...")
        val dockHint = TextView(this).apply {
            text = "内容由 AI 生成，请注意甄别重要信息"
            textSize = 11f
            setTextColor(dockText2)
            gravity = android.view.Gravity.CENTER
            setPadding(0, 5.dp2px(), 0, 0)
        }
        dockHintView = dockHint

```

```

        dock.addView(dockHint)
        root.addView(dock)

        // 工具按钮行 (全部隐藏——只保留发送;功能保留引用)
        val btnRow = LinearLayout(this).apply { orientation = Linea
rLayout.HORIZONTAL }
        val copyBtn = Button(this).apply {
            text = "复制日志"
            visibility = android.view.View.GONE
        }
        copyBtn.setOnClickListener {
            val cm = getSystemService(CLIPBOARD_SERVICE) as android
.content.ClipboardManager
            cm.setPrimaryClip(android.content.ClipData.newPlainText
("MOV日志", logView.text.toString()))
            android.widget.Toast.makeText(this, "日志已复制", android.w
idget.Toast.LENGTH_SHORT).show()
        }
        modeBtn = Button(this).apply {
            text = "模式:default"
            visibility = android.view.View.GONE
        }
        modeBtn.setOnClickListener { togglePermissionMode() }
        // D3:模型切换按钮 (循环切换注册表内模型)
        modelBtn = Button(this).apply {
            text = "模型:" + com.hermes.dsh.llm.ModelRegistry.current
().id

            visibility = android.view.View.GONE
        }
        modelBtn.setOnClickListener {
            val models = com.hermes.dsh.llm.ModelRegistry.list()
            if (models.size < 2) {
                appendLog("[模型] 仅一个模型,无需切换")
                return@setOnClickListener
            }
            val cur = com.hermes.dsh.llm.ModelRegistry.current()
            val next = models[(models.indexOfFirst { it.id == cur.i
d } + 1) % models.size]
            if (com.hermes.dsh.llm.ModelRegistry.use(next.id)) {
                modelBtn.text = "模型:" + next.id
                appendLog("[模型] 已切换 -> " + next.id + "(" + next.la
bel + ") ——后续对话生效")
            }
        }
        // C2 查验:上下文窗口切换 (1M 生产 / 5K 测试)
        val windowBtn = Button(this).apply {
            text = "窗口:1M"
            visibility = android.view.View.GONE
        }

```

```

// 测试开关：长按窗口按钮 = 强制审批走通知栏（前台也走，验证通知栏回答者）
windowBtn.setOnLongClickListener {
    forceNotification = !forceNotification
    appendLog("[审批] 渠道: " + if (forceNotification) "强制通知栏"
else "自动（前台弹窗/后台通知）")
    true
}
windowBtn.setOnClickListener {
    contextWindow = if (contextWindow >= 1000000) 5000 else
1000000
    val label = if (contextWindow >= 1000000) "1M" else (co
ntextWindow / 1000).toString() + "K"
    windowBtn.text = "窗口:" + label
    appendLog("上下文窗口已切换 -> " + label + " (85% 触发线 " + (cont
extWindow * 85 / 100) + " token)")
}
btnRow.addView(windowBtn, LinearLayout.LayoutParams(0, Linea
rLayout.LayoutParams.WRAP_CONTENT, 1f))
btnRow.addView(modelBtn, LinearLayout.LayoutParams(0, Linea
rLayout.LayoutParams.WRAP_CONTENT, 1f))
// 深浅模式切换按钮
val darkBtn = android.widget.Button(this).apply {
    text = "深色"
    textSize = 11f
    visibility = android.view.View.GONE
    setOnClickListener {
        isDark = !isDark
        text = if (isDark) "浅色" else "深色"
        // 消息流背景/文字随模式
        rootView?.setBackgroundColor(if (isDark) 0xFF14171B
.toInt() else 0xFFFFFFFF.toInt())
        logView.setTextColor(if (isDark) 0xFFE8EAE8.toInt()
else 0xFF1F2329.toInt())
        rebuildMessages()
    }
}
btnRow.addView(darkBtn, LinearLayout.LayoutParams(0, Linea
rLayout.LayoutParams.WRAP_CONTENT, 1f))
root.addView(btnRow)

// 启动应用深浅底色 (isDark 已在 onCreate 首段按 uiMode 初始化; 历史渲染各处按 isD
ark 取色, 无需 rebuildMessages)
rootView?.setBackgroundColor(if (isDark) 0xFF14171B.toInt()
else 0xFFFFFFFF.toInt())
logView.setTextColor(if (isDark) 0xFFE8EAE8.toInt() else 0x
FF1F2329.toInt())

// 侧边栏抽屉 (DrawerLayout:侧滑手势 + 遮罩自带)。面板 = WebView 装载 mov-vu
e

// SidebarNav 独立入口页 (三段结构由页面实现: WORKBENCH / HISTORY / profi

```

le+设置)

```

        val frame = android.widget.FrameLayout(this)
        frame.addView(root, android.widget.FrameLayout.LayoutParams
(
            android.widget.FrameLayout.LayoutParams.MATCH_PARENT,
            android.widget.FrameLayout.LayoutParams.MATCH_PARENT))
        val drawerLayout = androidx.drawerlayout.widget.DrawerLayout
t(this).apply {
            setScrimColor(0x66000000)
        }
        this.drawerLayout = drawerLayout
        drawerLayout.addView(frame, androidx.drawerlayout.widget.Dr
awerLayout.LayoutParams(
            androidx.drawerlayout.widget.DrawerLayout.LayoutParams.
MATCH_PARENT,
            androidx.drawerlayout.widget.DrawerLayout.LayoutParams.
MATCH_PARENT))
        // 抽屉面板:宽 60% 屏宽(沿用用户拍板收窄值)
        drawerWidthPx = (resources.displayMetrics.widthPixels * 0.6
0).toInt()
        drawerPanel = LinearLayout(this).apply {
            orientation = LinearLayout.VERTICAL
            // 面板背景铺满到屏幕顶(无暗带),底色与侧边栏页面 s0 一致(深 #0c0e12 / 浅 #f3f
4f7)
            val drawerNight = resources.configuration.uiMode and an
droid.content.res.Configuration.UI_MODE_NIGHT_MASK ==
                android.content.res.Configuration.UI_MODE_NIGHT_YES
            setBackgroundColor(if (drawerNight) 0xFF0C0E12.toInt()
else 0xFFFF3F4F7.toInt())
        }
        // Android 15 强制 edge-to-edge:面板按状态栏 inset 顶 padding(MOV A
I 品牌行不被状态栏压住)
        androidx.core.view.ViewCompat.setOnApplyWindowInsetsListene
r(drawerPanel!!) { v, insets ->
            val top = insets.getInsets(
                androidx.core.view.WindowInsetsCompat.Type.statusBa
rs() or
                    androidx.core.view.WindowInsetsCompat.Type.disp
layCutout(),
            ).top
            v.setPadding(0, top, 0, 0)
            insets
        }
        // 侧边栏 = WebView 抽屉(mov-vue SidebarNav 独立入口页)。
        // 桥白名单:room.* (房间列表/切换/新建) + ui.* (导航;ui.closePage 由 localH
andlers 关抽屉)
        val sidebarWeb = android.webkit.WebView(this)
        sidebarWebView = sidebarWeb
        sidebarWeb.settings.javaScriptEnabled = true
        sidebarWeb.settings.domStorageEnabled = true

```

```

        val sidebarAssetLoader = androidx.webkit.WebViewAssetLoader
        .Builder()
            .addPathHandler("/assets/", androidx.webkit.WebViewAssetLoader.AssetsPathHandler(this))
            .build()
        sidebarWeb.webViewClient = object : androidx.webkit.WebViewClientCompat() {
            override fun shouldOverrideUrlLoading(
                v: android.webkit.WebView,
                request: android.webkit.WebResourceRequest,
            ): Boolean {
                return WebLinkGuard.shouldOverrideUrlLoading(this@MainActivity, request)
            }

            override fun shouldInterceptRequest(
                v: android.webkit.WebView?,
                r: android.webkit.WebResourceRequest?,
            ): android.webkit.WebResourceResponse? {
                val url = r?.url ?: return null
                return sidebarAssetLoader.shouldInterceptRequest(url)
            }

            override fun onPageFinished(view: android.webkit.WebView, url: String) {
                applyPageTheme(view)
            }
        }
        sidebarWeb.addJavaScriptInterface(
            PagesBridge(
                webView = sidebarWeb,
                handlerProvider = { tool -> pageToolProvider(tool)
            },

            // 白名单: room. 房间区; ui. 页面导航 / 钉选槽位 / 输入框回填; market.status
            // 钉选工具禁用态灰化;

            // browser.open 打开有 UI 的工具入口 (browser.open 自带 http(s) 校验)

            allowedPrefixes = setOf("room.", "ui.", "market.status", "browser.open"),
            onClose = { runOnUiThread { closeRoomDrawer() } },
            localHandlers = mapOf(
                "ui.closePage" to {
                    runOnUiThread { closeRoomDrawer() }
                    mapOf("ok" to true)
                },
            ),
        ),
        "MovPageBridge",
    )
}

```

```

        sidebarWeb.loadUrl("https://appassets.androidplatform.net/a
ssets/pages/sidebar/sidebar.html")
        drawerPanel?.addView(sidebarWeb, LinearLayout.LayoutParams(
            LinearLayout.LayoutParams.MATCH_PARENT,
            LinearLayout.LayoutParams.MATCH_PARENT))
        drawerLayout.addView(drawerPanel, androidx.drawerlayout.wid
get.DrawerLayout.LayoutParams(
            drawerWidthPx,
            androidx.drawerlayout.widget.DrawerLayout.LayoutParams.
MATCH_PARENT,
            android.view.Gravity.START))

setContentViews(drawerLayout)
// 抽屉侧滑 vs 全面屏系统返回手势冲突:左缘 40dp 申请手势排除区(API 29+, 抽屉侧滑优先)
if (android.os.Build.VERSION.SDK_INT >= 29) {
    drawerLayout.post {
        val exclPx = 40.dp2px()
        drawerLayout.systemGestureExclusionRects = listOf(
            android.graphics.Rect(0, 0, exclPx, drawerLayout
t.height),
        )
    }
}
// 返回键: 抽屉开着先关抽屉, 关着走默认(退出)
onBackPressedDispatcher.addCallback(this, object : androidx
.activity.OnBackPressedCallback(true) {
    override fun handleOnBackPressed() {
        val dl = this@MainActivity.drawerLayout
        val panel = drawerPanel
        if (dl != null && panel != null && dl.isDrawerOpen(
panel)) {
            dl.closeDrawer(panel)
        } else {
            isEnabled = false
            onBackPressedDispatcher.onBackPressed()
        }
    }
})

// 软键盘 insets 处理(Android 11+):窗口 extend 到 system bars 下 +
手动应用 insets——
// 键盘弹出时底部 padding = IME 高度(输入框顶起、内容可滚动), 顶部 padding = 状态
栏(标题保持)
androidx.core.view.WindowCompat.setDecorFitsSystemWindows(w
indow, false)
rootView?.let { root ->
    androidx.core.view.ViewCompat.setOnApplyWindowInsetsLis
tener(root) { v, insets ->
        val sb = insets.getInsets(androidx.core.view.Window
InsetsCompat.Type.statusBars())

```

```

        val ime = insets.getInsets(androidx.core.view.WindowInsetsCompat.Type.ime())
        v.setPadding(sb.left, sb.top, sb.right, ime.bottom)
        insets
    }
}

// F4:凭据加密存储初始化 + 恢复已保存的 API key(keyInput 已初始化)
credentials = CredentialStore(this)
// M1 房间存储:启动 scan + 首房间迁移(android-session 无 meta → title
从历史派生)
roomStore = com.hermes.dsh.session.RoomStore(this).apply {
    scan()
}
// 孤儿子会话清理(parent 房间已不存在的 session-* 目录;IO 操作异步跑,不卡启动)
scope.launch(kotlinx.coroutines.Dispatchers.IO) {
    try {
        val ids = (roomStore?.listRooms()?.map { it.id } ?:
emptyList()).toSet()
        val deleted = com.hermes.dsh.subagent.SubagentIndex
.cleanupOrphans(File(filesDir, "sessions"), ids)
        if (deleted.isNotEmpty()) appendLog("[清理] 孤儿子会话: "
+ deleted.joinToString(", "))
    } catch (e: Exception) {
        android.util.Log.w("MOV-Boot", "孤儿子会话清理失败: ${e.mes
sage}")
    }
}
val savedKey = credentials.get("deepseek_key")
if (!savedKey.isNullOrEmpty()) {
    keyInput.setText(savedKey)
} else {
    // 迁移旧明文(首次升级):读 mov_prefs 明文 → 加密存储 → 删除明文
    val legacy = getSharedPreferences("mov_prefs", MODE_PRI
VATE).getString("deepseek_key", "")
    if (!legacy.isNullOrEmpty()) {
        credentials.put("deepseek_key", legacy)
        getSharedPreferences("mov_prefs", MODE_PRIVATE).edi
t().remove("deepseek_key").apply()
        keyInput.setText(legacy)
        appendLog("[F4] 已迁移凭据到 Keystore 加密存储(明文已删除)")
    }
}

// 请求运行时权限:通知权限启动即需(后台审批通知是核心链路);
// 蓝牙权限延迟到首次调用蓝牙工具时再申请(启动不打扰用户)
if (android.os.Build.VERSION.SDK_INT >= 31) {
    requestPermissions(arrayOf(
        android.Manifest.permission.POST_NOTIFICATIONS,
    ), 100)
}

```

```

    }

    // 初始化 store + 会话持久化 (B3:JSONL 落盘 + 崩溃恢复)
    store = SessionStore()
    persistence = run {
        val root = File(filesDir, "sessions").absolutePath
        val backend = com.hermes.dsh.session.persistence.jsonl.
JsonlSessionPersistence(root)
        val coord = com.hermes.dsh.session.persistence.Persiste
nceCoordinator(backend, store)
        coord.installWritePath()
        // F2:FTS5 索引 (日志为权威, 索引可重建派生数据——异步同步 + 崩溃重建兜底)
        // F2 底座:打包 SQLite FTS5 (osmerion 3.49.2, 系统无 fts5 模块) ——
search.db 独立于 B3
        fts5 = com.hermes.dsh.sessionquery.Fts5QueryEngine(
            com.osmerion.android.database.sqlite.SQLiteDatabase
.openOrCreateDatabase(
                File(filesDir, "search.db").absolutePath, null,
            ),
            com.hermes.dsh.session.persistence.jsonl.JsonlSessi
onPersistence(File(filesDir, "sessions").absolutePath),
        )
        coord.onIndexed = { sid, events ->
            android.util.Log.i("MOV-F2", "onIndexed ${events.si
ze} 事件")

            kotlinx.coroutines.GlobalScope.launch {
                try { events.forEach { fts5!!.indexEvent(sid, i
t) } } catch (e: Exception) { android.util.Log.e("MOV-F2", "索引失败: $
{e.message}") }
            }
        }
        coord
    }

    // 启动恢复异步化:磁盘 IO (persistence.load/fts5.rebuild) 不在主线程 runB
locking (ANR 风险) ——
    // 先挂加载遮罩, 协程内仅 IO 段下 Dispatchers.IO, 其余(store/render/日志)保持
主线程原顺序
    showRoomLoading()
    scope.launch {
        // M3:目标恢复房间 (try/catch 共用——catch 只清理该房间, 不删全部)
        val targetId = roomStore?.listRooms()?.firstOrNull()?.i
d ?: "android-session"
        val restored: com.hermes.dsh.session.Session = try {
            // M3:启动恢复最新房间 (updatedAt 倒序首位); 无任何房间 → android-se
ssion 兜底 (含 M1 迁移)
            val latest = roomStore?.listRooms()?.firstOrNull()
            val s: com.hermes.dsh.session.Session = if (latest
!= null && roomStore?.hasLog(latest) == true) {
                // 最新房间有日志:Restored 恢复 (含未闭合轮修复)
                val inspection = withContext(Dispatchers.IO) {

```

```

persistence!!.load(SessionId(latest.id)) }
        appendLog("恢复会话: " + latest.id + "(" + inspection.events.size + " 事件)")
        val prepared = store.prepare(
            SessionId(latest.id),
            com.hermes.dsh.session.PrepareSessionOptions.Restored(
                com.hermes.dsh.session.RestoredSessionOptions(
                    seed = inspection.events,
                    meta = inspection.meta,
                ),
            ),
        )
        sessionRelease = store.enter(prepared)
        store.announce(prepared)
        prepared
    } else if (latest != null) {
        // 最新房间无日志 (blank 新对话): Create—lazy 首个消息才落盘
        appendLog("恢复会话: " + latest.id + "(新房间, 无日志)")
        val prepared = store.prepare(SessionId(latest.id))

        sessionRelease = store.enter(prepared)
        store.announce(prepared)
        prepared
    } else {
        // 无房间 (首次 / 纯旧数据): android-session 恢复 + M1 迁移
        val inspection = withContext(Dispatchers.IO) {
            persistence!!.load(SessionId("android-session")) }
        appendLog("恢复会话: android-session(" + inspection.events.size + " 事件)")
        val prepared = store.prepare(
            SessionId("android-session"),
            com.hermes.dsh.session.PrepareSessionOptions.Restored(
                com.hermes.dsh.session.RestoredSessionOptions(
                    seed = inspection.events,
                    meta = inspection.meta,
                ),
            ),
        )
        sessionRelease = store.enter(prepared)
        store.announce(prepared)
        // M1 首房间迁移: android-session → 房间 meta (title 历史派生)
        roomStore?.ensureFirstRoom(prepared)
        prepared
    }
    // B3 UI 历史渲染: 把恢复的消息画进对话区 (遮罩由 onSettled / 超时兜底撤)

```

```

        renderHistory(s)
        // F2:启动重建搜索索引(历史进索引——rebuild 是唯一入口,新事件靠 onIndexe
d 增量)

        try {
            withContext(Dispatchers.IO) { fts5?.rebuild(s.i
d, s.events) }

            android.util.Log.i("MOV-F2", "启动重建索引 ${s.event
s.size} 事件")
        } catch (e: Exception) {
            android.util.Log.e("MOV-F2", "启动重建索引失败: ${e.me
ssage}")
        }
        s
    } catch (e: Exception) {
        android.util.Log.e("MOV-Persist", "恢复失败,隔离该房间后新建:
" + e.message)

        appendLog("[错误] 恢复失败,已重建会话: " + e.message)
        // M3 修复:只处理出问题的那个房间(一个房间损坏不能删光全部会话)
        // 隔离而非删除:目录 rename 为 <id>.corrupt-<时间戳>(rename 失
败才 deleteRecursively),
        // 再调 deleteRoom 清 RoomStore meta 缓存(目录已隔离,其物理删除落
空)

        var quarantined: String? = null
        try {
            val root = File(filesDir, "sessions")
            val ts = java.text.SimpleDateFormat("yyyyMMdd-H
Hmss", java.util.Locale.US).format(java.util.Date())
            root.listFiles()?.forEach { cwd ->
                if (cwd.isDirectory) {
                    val dir = File(cwd, targetId)
                    if (dir.exists()) {
                        val dst = File(cwd, targetId + ".co
rrupt-" + ts)

                        if (dir.renameTo(dst)) quarantined
= dst.name else dir.deleteRecursively()
                    }
                }
            }
            roomStore?.deleteRoom(targetId)
        } catch (_: Exception) {
        }
        if (quarantined != null) {
            appendLog("[提示] 会话数据已隔离到 " + quarantined + "(s
essions 目录下),可人工恢复")
        }
        hideRoomLoading()
        store.create(SessionId("android-session"))
    }
    session = restored
    appendLog("会话: " + restored.id.value)

```

```

        // D4：启动恢复审批策略（日志为唯一权威）——必须在此处调用：
        // 恢复协程完成前 session 为 null、onCreate 同步段 permissionGuard
未创建，旧调用点必然 no-op。
        // 本协程回主线程时 onCreate 同步段已执行完，permissionGuard 一定就绪。
        restoreApprovalPolicy()
        updateRoomTitle()
    }
    // 通知栏审批：动态注册按钮广播接收器
    com.hermes.dsh.tools.NotificationAnswerer.register(this)
    appendLog("=== MOV 初始化完成 ===")
    // 会话 id 日志在上方恢复协程完成后打印（session 异步赋值，此处可能还是 null）

    // MCP 工具注册表（供服务器 + agent 共用）
    mcpHandlers = LinkedHashMap<String, (Map<String, Any?>) ->
Any?>()
    val queryTools = com.hermes.mov.tools.MovQueryTools { store
        .list() }
    mcpHandlers["mov.roomList"] = { queryTools.roomList() }
    mcpHandlers["mov.subagentList"] = { args ->
        val roomId = args["roomId"] as? String
        val children = com.hermes.dsh.subagent.SubagentIndex.lis
tChildren(File(filesDir, "sessions"), roomId?.ifEmpty { null })
        mapOf(
            "ok" to true,
            "count" to children.size,
            "children" to children.map {
                mapOf(
                    "childId" to it.childId,
                    "parentId" to it.parentId,
                    "depth" to it.depth,
                    "createdAt" to it.createdAt,
                    "eventCount" to it.eventCount,
                )
            },
        )
    }
    mcpHandlers["mov.journalTail"] = { args ->
        var roomId = args["roomId"] as? String ?: ""
        // 默认房间：roomId 为空时取第一个会话，避免"房间不存在："
        if (roomId.isEmpty()) {
            roomId = store.list().firstOrNull()?.id?.value ?: ""
        }
        queryTools.journalTail(
            roomId = roomId,
            afterSeq = (args["afterSeq"] as? Number)?.toInt() ?
: 0,
            count = (args["count"] as? Number)?.toInt() ?: 20,
        )
    }
}

```

```

        view.evaluateJavascript(js, null)
    }
    refit()
}

override fun onReceivedError(v: android.webkit.
WebView, request: android.webkit.WebResourceRequest, error: android
x.webkit.WebResourceErrorCompat) {
    // 主帧加载失败 (渲染页不可达) → 标记降级, 后续走原生 (不白屏)
    if (request.isMainFrame && !markstreamBr
oken) {

        markstreamBroken = true
        appendLog("[提示] markstream 渲染页加载失败, 降级为
内置渲染")

    }
}

loadUrl("https://appassets.androidplatform.net/asse
ts/markstream/room.html")
}
liveMarkstreams.add(this) // onDestroy/LRU 驱逐统一销毁
}

/** 主线程执行页面 API (未就绪排队)。 */
private fun eval(js: String) {
    runOnUiThread {
        if (!ready) { pending.add(js); return@runOnUiThread

        view.evaluateJavascript(js) { refit() }
    }
}

/** 整房历史原子替换 (json: [{role,md}...] 的 JSON 字符串)。 */
fun setHistory(json: String) = eval("__room.setHistory(" +
jsString(json) + ")")

/** 用户气泡入页 (右对齐 HTML 气泡, 与历史同款); imageUrl 可选 (照片消息)。 */
fun addUser(text: String, imageUrl: String? = null) =
    eval("__room.addUser(" + jsString(text) + ", " + (if (i
mageUrl != null) jsString(imageUrl) else "null") + ")")

fun beginStream() = eval("__room.beginStream()")
fun appendChunk(text: String) = eval("__room.appendChunk("
+ jsString(text) + ")")
fun endStream() = eval("__room.endStream()")

/** 高度自适应 (延迟防频繁触发)。 */
fun refit() {
    view.postDelayed({ fitWebViewHeight(view) }, 60)
}
}

```

```

/** 当前房间的房间页（实例缓在容器子 view 的 tag 里——缓存房间挂回时随容器找回）。 */
private fun getOrCreateRoomView(): MarkstreamView {
    val c = mdContainer
    if (c != null) {
        for (i in 0 until c.childCount) {
            val t = c.getChildAt(i).tag
            if (t is MarkstreamView) return t
        }
    }
    val mv = MarkstreamView()
    mv.view.tag = mv
    c?.addView(mv.view)
    return mv
}

/** WebView 内容高度自适应（WRAP_CONTENT 被系统限高，加载后按内容测量）。 */
private fun fitWebViewHeight(wv: android.webkit.WebView) {
    val lp = wv.layoutParams as? android.widget.LinearLayout.LayoutParams ?: return
    if (wv.width <= 0) return
    wv.measure(
        android.view.View.MeasureSpec.makeMeasureSpec(wv.width,
            android.view.View.MeasureSpec.EXACTLY),
        android.view.View.MeasureSpec.makeMeasureSpec(0, android.view.View.MeasureSpec.UNSPECIFIED),
    )
    val h = wv.measuredHeight
    if (h > 0) {
        lp.height = h
        wv.layoutParams = lp
    }
}

/** 递归销毁容器里的 WebView（缓存驱逐 / 删房间时释放渲染进程内存）。 */
private fun destroyWebViewsIn(v: android.view.View) {
    if (v is android.webkit.WebView) {
        v.destroy()
        return
    }
    if (v is android.view.ViewGroup) {
        for (i in v.childCount - 1 downTo 0) destroyWebViewsIn(v.getChildAt(i))
    }
}

/** 进入房间：渲染好前显示加载遮罩（不露“只有用户消息 / 渲染过程”）。 */
private fun showRoomLoading() {
    val root = rootView ?: return

```

```

        val parent = root.parent as? android.view.ViewGroup ?: return
        if (roomLoadingView == null) {
            roomLoadingView = android.widget.FrameLayout(this).apply {
                setBackgroundColor(if (isDark) 0x8814171B.toInt() else 0x88FFFFFF.toInt())
                val content = android.widget.LinearLayout(this@MainActivity).apply {
                    orientation = android.widget.LinearLayout.VERTICAL
                    gravity = android.view.Gravity.CENTER
                    addView(android.widget.ProgressBar(this@MainActivity))
                    addView(android.widget.TextView(this@MainActivity).apply {
                        text = "正在加载房间..."
                        setPadding(0, 12, 0, 0)
                        setTextColor(if (isDark) 0xFFE8EAE8.toInt() else 0xFF1F2329.toInt())
                    })
                }
                addView(content, android.widget.FrameLayout.LayoutParams(
                    android.view.ViewGroup.LayoutParams.WRAP_CONTENT,
                    android.view.ViewGroup.LayoutParams.WRAP_CONTENT,
                    android.view.Gravity.CENTER,
                ))
            }
        }
        if (roomLoadingView?.parent == null) {
            parent.addView(roomLoadingView, android.widget.FrameLayout.LayoutParams(
                android.view.ViewGroup.LayoutParams.MATCH_PARENT,
                android.view.ViewGroup.LayoutParams.MATCH_PARENT,
            ))
        }

        private fun hideRoomLoading() {
            roomLoadingView?.let { (it.parent as? android.view.ViewGroup)?.removeView(it) }
        }

        private inner class StreamMdRenderer(private val host: android.widget.LinearLayout? = null) {
            private val machine = StreamMdMachine()
            private var tableAligns: List<Int> = emptyList()

```

```

        private var tableHost: android.widget.HorizontalScrollView?
= null
        private var rawView: android.widget.TextView? = null
        private var rawPending = ""
        private var rawScheduled = false
        private val mainHandler = android.os.Handler(android.os.Looper.getMainLooper())
        private val container: android.widget.LinearLayout?
            get() = host?: mdView.parent as? android.widget.LinearLayout

        val hasContent: Boolean get() = machine.hasContent
        val fullText: StringBuilder get() = machine.fullText

        /** chunk 文本到达(主线程):NBSP → 普通空格后交状态机。 */
        fun accumulate(text: String) {
            if (machine.isFinished || text.isEmpty()) return
            val clean = text.replace('\u00A0', ' ')
            execute(machine.accumulate(clean))
            scheduleRaw()
        }

        /** 流结束/中断:剩余缓冲渲染,补复制图标,清理空 rawView。 */
        fun finish() {
            if (machine.isFinished) return
            mainHandler.removeCallbacksAndMessages(null)
            execute(machine.finish())
            flushRawNow()
            if (machine.hasContent && machine.fullText.isNotBlank()) {
                val copyReply = android.widget.ImageView(this@MainActivity).apply {
                    setImageResource(R.drawable.ic_copy)
                    setPadding(4, 2, 4, 2)
                    setOnClickListener {
                        try {
                            val cm = getSystemService(android.content.Context.CLIPBOARD_SERVICE) as android.content.ClipboardManager
                            cm.setPrimaryClip(android.content.ClipData.newPlainText("reply", machine.fullText.toString()))
                            android.widget.Toast.makeText(this@MainActivity, "已复制", android.widget.Toast.LENGTH_SHORT).show()
                        } catch (_: Exception) {
                        }
                    }
                }
                copyReply.layoutParams = android.widget.LinearLayout.LayoutParams(
                    android.widget.LinearLayout.LayoutParams.WRAP_CONTENT,

```

```

        android.widget.LinearLayout.LayoutParams.WRAP_C
CONTENT,

        ).apply { gravity = android.view.Gravity.START }
        container?.let { it.addView(copyReply) } // 消息流末尾 (
回复后)

    }
    scrollDown()
    // 收尾清理：无内容的空 rawView 移除 (防残留空白块)
    val rv = rawView
    if (rv != null && rv.text.isNullOrEmpty()) {
        container?.removeView(rv)
        rawView = null
    }
}

/** 执行动作流 (顺序保持——Raw 追加原文, Render* 排版替换)。markstream 模式只保留 R
aw (打字机), 不排版。 */
private fun execute(actions: List<StreamMdMachine.Action>)
{
    for (a in actions) {
        when (a) {
            is StreamMdMachine.Action.Raw -> rawAppend(a.te
xt)

            else -> when (a) {
                is StreamMdMachine.Action.RenderText -> {
                    val mw = markwon ?: return
                    insertRendered(mdTextBlock(a.md, mw))
                }
                is StreamMdMachine.Action.RenderCode ->
                    insertRendered(buildCodeBlockView(CodeM
d(a.lang, a.code, 0, 0)))
                is StreamMdMachine.Action.RenderMath ->
                    insertRendered(buildMathView(MathMd(a.l
atex, 0, 0)))
                is StreamMdMachine.Action.StartTable -> sta
rtTable(a.headerLine, a.sepLine)
                is StreamMdMachine.Action.TableRow -> {
                    val host = tableHost
                    val tl = host?.getChildAt(0) as? androi
d.widget.TableLayout

                    val mw = markwon
                    if (host != null && tl != null && mw !=
null) {
                        tl.addView(buildRow(MdTableParse.pa
rseRow(a.line), tableAligns, isHeader = false, mw))
                        tl.addView(bottomLine(1, if (isDark
) 0x262A2F36.toInt() else 0x1AE3E7EB.toInt()))
                    }
                }
            }
        }
    }
}

```

```

    }
    }
}

/** 表格确认：表头+分隔行 → 表格视图（含表头原文的 rawView 由 insertRendered 移除
，不重复显示）。 */
private fun startTable(headerLine: String, sepLine: String)
{
    tableAligns = MdTableParse.parseAlign(sepLine)
    val mw = markwon ?: return
    val t1 = android.widget.TableLayout(this@MainActivity).
apply {
    layoutParams = android.widget.TableLayout.LayoutParams(
        android.widget.TableLayout.LayoutParams.WRAP_CO
CONTENT,
        android.widget.TableLayout.LayoutParams.WRAP_CO
CONTENT,
    )
    background = android.graphics.drawable.GradientDraw
able().apply {
        cornerRadius = 8f
        setColor(if (isDark) 0xFF1B1F25.toInt() else 0x
FFF2F4F7.toInt())
        setStroke(1, if (isDark) 0xFF2A2F36.toInt() els
e 0xFFE3E7EB.toInt())
    }
    setPadding(12, 8, 12, 8)
}
    t1.addView(buildRow(MdTableParse.parseRow(headerLine),
tableAligns, isHeader = true, mw))
    t1.addView(bottomLine(2, if (isDark) 0xFF3A4256.toInt()
else 0xFFE3E7EB.toInt()))
    val hsv = android.widget.HorizontalScrollView(this@Main
Activity).apply {
        layoutParams = android.widget.LinearLayout.LayoutPa
rams(
            android.widget.LinearLayout.LayoutParams.MATCH_
PARENT,
            android.widget.LinearLayout.LayoutParams.WRAP_C
CONTENT,
        ).apply { topMargin = 4; bottomMargin = 4 }
        isHorizontalScrollBarEnabled = false
    }
    hsv.addView(t1)
    tableHost = hsv
    insertRendered(hsv)
}

```

```

    /** 渲染视图插入 rawView 前（原文被排版替换，rawView 移除重建）；rv 空时插容器末尾——消
息流追加语义。 */
    private fun insertRendered(v: android.view.View) {
        flushRawNow()
        val c = container ?: return
        val rv = rawView
        if (rv != null) {
            val idx = c.indexOfChild(rv)
            c.addView(v, idx)
            c.removeView(rv)
            rawView = null
        } else {
            c.addView(v)
        }
        scrollDown()
    }

    /** 进行中原文（打字机）：缓冲待节流刷新。 */
    private fun rawAppend(text: String) {
        rawPending += text
        scheduleRaw()
    }

    private fun scheduleRaw() {
        if (rawScheduled || machine.isFinished) return
        rawScheduled = true
        mainHandler.postDelayed({ flushRawNow() }, 40)
    }

    private fun flushRawNow() {
        rawScheduled = false
        if (rawPending.isEmpty()) return
        val v = rawView ?: createRawView()
        v.append(rawPending)
        rawPending = ""
        scrollDown()
    }

    private fun createRawView(): android.widget.TextView {
        val tv = android.widget.TextView(this@MainActivity).app
ly {
            textSize = 15f
            setLineSpacing(0f, 1.45f)
            setTextColor(if (isDark) 0xFFE8EAE8.toInt() else 0x
FF1F2329.toInt())
            setTextIsSelectable(true)
        }
        tv.layoutParams = android.widget.LinearLayout.LayoutPar
ams (
            android.widget.LinearLayout.LayoutParams.MATCH_PARE

```

```

NT,
        android.widget.LinearLayout.LayoutParams.WRAP_CONTE
NT,
    )
    val c = container ?: return tv
    c.addView(tv) // 消息流末尾(插入 mdView+1 会恒位导致块倒序)
    rawView = tv
    return tv
}

private fun scrollDown() {
    try {
        val p = mdView.parent as? android.view.View ?: retu
rn
        (p.parent as? android.widget.ScrollView)?.fullScrol
l(android.view.View.FOCUS_DOWN)
    } catch (_: Exception) {
    }
}
}

/** dsh 对齐:纯 HTTP(S) URL 行内代码 → 链接(命令/路径/部分 URL 保持胶囊不可点)。 */
/**
 * 反引号内的网址 → markdown 链接(Markwon 渲染 URLSpan → applyLinkClick 显
式点击)。
 * ① 带协议(https://...) → 原样转链接
 * ② 裸域名(www. 开头或三级+域名,如 www.cctv.com / api.deepseek.com) → 补 h
ttp:// 转链接
 * 保守判定防误判:单点域名(main.py/config.json 等文件路径)、版本号(v1.2.3)不转
 */
/** 渲染 AI 消息为 md 排版:commonmark AST 手动遍历——表格块 → TableLayout,其余
→ Markwon。 */
/** 渲染 AI 消息为 md 排版:正则提取表格(富内容+对齐) → TableLayout;其余 Markwon 段落
。 */
private fun appendAiMd(text: String, withIcon: Boolean = true)
{
    try {
        var t = text.trim()
        // 防御:AI 回复带 [用户]/[AI] 角色前缀(模型格式怪癖)——剥离
        t = t.removePrefix("[用户] ").removePrefix("[AI] ").remov
ePrefix("[用户]").removePrefix("[AI]")
        if (t.isEmpty()) return
        val mw = markwon ?: return
        val container = mdView.parent as? android.widget.Linear
Layout ?: return
        // 整条回复包进 AI 卡片(文本/表格/代码/数学块都进卡片,对齐原型 .msg.ai)
        val card = createAiCard()
        container.addView(card)
        // URL 代码 → 链接(Markwon 原生 URLSpan 可点击——dsh URL 代码 chro
me)

```

```

        val lines = com.mov.android.md.MdPreprocess.promoteUrlCodes(t).lines().toMutableList()
        val tables = extractTables(lines)
        // 表格占位清空 (保留行号——按位置分段)
        for (tb in tables) {
            for (k in tb.start until tb.end) lines[k] = ""
        }
        // 代码块提取 (`` ` 块 → 独立 View:语言横幅 + 高亮 + 长按复制)
        val codes = extractCodeBlocks(lines)
        for (cb in codes) {
            for (k in cb.start until cb.end) lines[k] = ""
        }
        // 数学块提取 ($$...$$ → 独立数学视图, jlatexmath 渲染)
        val maths = extractMathBlocks(lines)
        for (mb in maths) {
            for (k in mb.start until mb.end) lines[k] = ""
        }
        // 按位置切分:文本段 → 表格/代码/数学 → 文本段...
        val parts = mutableListOf<Pair<Int, Any>>() // 0=text 1
=table 2=code 3=math
        val blocks = (tables.map { MdBBlock(1, it, it.start) } +
            codes.map { MdBBlock(2, it, it.start) } +
            maths.map { MdBBlock(3, it, it.start) }).sortedBy {
it.start }
        var cursor = 0
        for ((kind, payload, start) in blocks) {
            val textPart = lines.subList(cursor, start).joinToString("\n").trim('\n')
            if (textPart.isNotBlank()) parts.add(0 to textPart)
            parts.add(kind to payload)
            cursor = when (payload) {
                is TableMd -> payload.end
                is CodeMd -> payload.end
                is MathMd -> payload.end
                else -> start
            }
        }
        val tail = lines.subList(cursor, lines.size).joinToString("\n").trim('\n')
        if (tail.isNotBlank()) parts.add(0 to tail)
        // 渲染 (顺序保留——追加末尾, 与用户气泡同侧, 一问一答按时间排列)
        for ((kind, payload) in parts) {
            when (kind) {
                1 -> {
                    // 宽表格可横向滚动 (dsh tableScroll:overflow-x auto)
                    val hsv = android.widget.HorizontalScrollView(this).apply {
                        layoutParams = android.widget.LinearLayout
out.layoutParams(

```

```

        android.widget.LinearLayout.LayoutParams.MATCH_PARENT,
        android.widget.LinearLayout.LayoutParams.WRAP_CONTENT,
        4
    ).apply { topMargin = 4; bottomMargin = 4 }

    isHorizontalScrollBarEnabled = false
}
hsv.addView(buildTableViews(payload as TableMd, mw))

card.addView(hsv)
}
2 -> card.addView(buildCodeBlockView(payload as CodeMd))
3 -> card.addView(buildMathView(payload as MathMd))

else -> card.addView(mdTextBlock(payload as String, mw))

}

// 整条回复复制图标（简约——点击复制 + Toast）
if (withIcon) appendCopyIcon(text, card)
(container.parent as? android.widget.ScrollView)?.fullScroll(android.view.View.FOCUS_DOWN)
} catch (e: Exception) {
    android.util.Log.e("MOV-MD", "appendAiMd 异常: " + (e.message ?: ""))
}

}

/** 回复后追加复制图标（markstream/markwon 共用）。 */
private fun appendCopyIcon(text: String, host: android.widget.LinearLayout? = null) {
    val container = host ?: mdView.parent as? android.widget.LinearLayout ?: return
    val copyReply = android.widget.ImageView(this).apply {
        setImageResource(R.drawable.ic_copy)
        setPadding(4, 2, 4, 2)
        setOnClickListener {
            try {
                val cm = getSystemService(android.content.Context.CLIPBOARD_SERVICE) as android.content.ClipboardManager
                cm.setPrimaryClip(android.content.ClipData.newPlainText("reply", text))
                android.widget.Toast.makeText(this@MainActivity, "已复制", android.widget.Toast.LENGTH_SHORT).show()
            } catch (_: Exception) {}
        }
    }
}
}
}

```

```

        copyReply.layoutParams = android.widget.LinearLayout.LayoutParams(
            android.widget.LinearLayout.LayoutParams.WRAP_CONTENT,
            android.widget.LinearLayout.LayoutParams.WRAP_CONTENT,
        ).apply { gravity = android.view.Gravity.START }
        container.addView(copyReply) // 追加末尾 (与消息块同侧)
    }

    /** Markwon 段落块 (非表格 md 内容; dsh 对齐: 块间距 8px + 链接 seal 色)。 */
    /**
     * 链接显式点击处理: URLSpan → ClickableSpan (显式 ACTION_VIEW + try-catch
     + Toast)。
     * 不依赖 URLSpan 默认 onClick (无异常处理——Intent 无浏览器匹配时静默失败/崩溃)。
     * 样式: seal 蓝 + 下划线 (明确可点)。
     */
    private fun applyLinkClick(spannable: android.text.Spannable) {
        val urls = spannable.getSpans(0, spannable.length, android.
text.style.URLSpan::class.java)
        for (span in urls) {
            val url = span.url
            val start = spannable.getSpanStart(span)
            val end = spannable.getSpanEnd(span)
            spannable.removeSpan(span)
            spannable.setSpan(
                object : android.text.style.ClickableSpan() {
                    override fun onClick(widget: android.view.View)
{
                        try {
                            val uri = android.net.Uri.parse(url)
                            val host = uri.host ?: ""
                            val port = uri.port
                            // markstream 渲染页 (本机 8799) → 内置 WebView
;其余走系统浏览器

                            if ((host == "127.0.0.1" || host == "lo
calhost") && port == 8799) {
                                widget.context.startActivity(
                                    android.content.Intent(
                                        widget.context,
                                        MarkstreamViewActivity::cla
ss.java,

                                        ).putExtra("url", url),
                                    )
                            } else {
                                widget.context.startActivity(
                                    android.content.Intent(
                                        android.content.Intent.ACTI
ON_VIEW,

                                        uri,

                                    ),
                                )
                            }
                        }
                    }
                }
            )
        }
    }

```

```

    }
    } catch (e: Exception) {
        android.widget.Toast.makeText(
            this@MainActivity,
            "无法打开链接: $url",
            android.widget.Toast.LENGTH_SHORT,
        ).show()
    }
}

override fun updateDrawState(ds: android.text.TextPaint) {
    super.updateDrawState(ds)
    ds.color = 0xFF2E6BE6.toInt()
    ds.isUnderlineText = true
}

},
start, end,
android.text.Spanned.SPAN_EXCLUSIVE_EXCLUSIVE,
)
}
}

private fun mdTextBlock(text: String, mw: io.noties.markwon.Markwon): TextView {
    // 纯文本数学符号(√ 等) → LaTeX → 提取渲染(解决字形缺失)
    val converted = com.mov.android.math.MathExtractor.convertPlainMath(text)
    // 行内数学 $...$:分段构建占位符 → Markwon 渲染 → ImageSpan 替换(数学位图)
    val segs = com.mov.android.math.MathExtractor.extract(converted)
        .filter { it.latex.length <= com.mov.android.math.MathRendererer.MAX_LATEX_LEN }
    val placeholder = "\uE000"
    var marked = converted
    if (segs.isNotEmpty()) {
        val sb = StringBuilder()
        var cursor = 0
        for ((idx, s) in segs.withIndex()) {
            sb.append(converted, cursor, s.start)
            sb.append(placeholder).append(idx.toString(36))
            cursor = s.end
        }
        sb.append(converted, cursor, converted.length)
        marked = sb.toString()
    }
    val tv = TextView(this).apply {
        textSize = 15f
        setLineSpacing(0f, 1.45f)
        setPadding(0, 2.dp2px(), 0, 2.dp2px())
    }
}

```

```

        setColor(if (isDark) 0xFFE8EAE.toInt() else 0xFF1F
2329.toInt())
        // 链接/URL 代码可点击 (LinkMovementMethod——长按选择牺牲, 换取点击)
        movementMethod = android.text.method.LinkMovementMethod
.getInstance()
        val spannable = mw.toMarkdown(marked) as android.text.S
pannable

        // 数学占位符 → ImageSpan (渲染失败/超长则保留占位文本——几乎不可见, 回退安全)
        if (segs.isNotEmpty()) {
            val mathColor = if (isDark) 0xFFE8EAE.toInt() else
0xFF1F2329.toInt()
            val textSizePx = 15f * resources.displayMetrics.den
sity

            for ((idx, s) in segs.withIndex()) {
                val ph = placeholder + idx.toString(36)
                val p0 = spannable.toString().indexOf(ph)
                if (p0 < 0) continue
                val drawable = com.mov.android.math.MathRender
r.renderLatex(
                    s.latex, textSizePx, mathColor, resources.d
isplayMetrics.density,
                )
                if (drawable != null) {
                    spannable.setSpan(
                        android.text.style.ImageSpan(drawable,
android.text.style.ImageSpan.ALIGN_BASELINE),
                        p0, p0 + ph.length,
                        android.text.Spanned.SPAN_EXCLUSIVE_EXC
LUSIVE,
                    )
                }
            }
        }
        // 链接显式点击 (seal 蓝 + 下划线 + try-catch Toast——替代 URLSpan 默
认静默)
        applyLinkClick(spannable)
        // 删除线文字 → 红色 (删除语义——划线跟随 textColor)
        for (span in spannable.getSpans(0, spannable.length, an
droid.text.style.StrikethroughSpan::class.java)) {
            spannable.setSpan(
                android.text.style.ForegroundColorSpan(0xFFE05E
65.toInt()),
                spannable.getSpanStart(span),
                spannable.getSpanEnd(span),
                android.text.Spanned.SPAN_EXCLUSIVE_EXCLUSIVE,
            )
        }
        this.text = spannable
    }
    tv.layoutParams = android.widget.LinearLayout.LayoutParams(

```

```

        android.widget.LinearLayout.LayoutParams.MATCH_PARENT,
        android.widget.LinearLayout.LayoutParams.WRAP_CONTENT,
    ).apply { topMargin = 2.dp2px(); bottomMargin = 2.dp2px() }
    return tv
}

/** 表格数据（保留 md 源——单元格富内容渲染）。 */
private data class TableMd(
    val header: List<String>,
    val rows: List<List<String>>,
    val aligns: List<Int>, // 0=left 1=center 2=right
    val start: Int,
    val end: Int,
)

/** 分块（渲染顺序：kind 0=text 1=table 2=code 3=math）。 */
private data class Mdblock(
    val kind: Int,
    val payload: Any,
    val start: Int,
)

private data class CodeMd(
    val lang: String,
    val content: String,
    val start: Int,
    val end: Int,
)

/** 数学块（$$...$$, LaTeX 内容）。 */
private data class MathMd(
    val latex: String,
    val start: Int,
    val end: Int,
)

/** 数学块提取（`$$...$$` / ``\[...\]``：同行闭合或多行累积到闭合；悬空开标记不提取）。 */
private fun extractMathBlocks(lines: List<String>): List<MathMd>
> {
    val out = mutableListOf<MathMd>()
    var i = 0
    while (i < lines.size) {
        val t = lines[i].trim()
        val open = if (t.startsWith("$") "$" else if (t.startsWith("\[")) "\[" else null
        if (open != null && t.length >= open.length) {
            val close = if (open == "$" "$" else "\]"
            val rest = t.removePrefix(open)
            if (rest.endsWith(close)) {
                out += MathMd(rest.removeSuffix(close).trim(),

```

```

i, i + 1)

        i++
        continue
    }
    val sb = StringBuilder()
    var j = i
    var closed = false
    if (rest.isNotEmpty()) sb.append(rest).append('\n')
    j++
    while (j < lines.size) {
        val l = lines[j]
        if (l.trim().endsWith(close)) {
            sb.append(l.removeSuffix(close).trimEnd())
            closed = true
            break
        }
        sb.append(l).append('\n')
        j++
    }
    if (closed) {
        out += MathMd(sb.toString().trim(), i, j + 1)
        i = j + 1
        continue
    }
}
i++
}
return out
}

```

/** 正则提取代码块：```lang ... ```。 */

```

private fun extractCodeBlocks(lines: List<String>): List<CodeMd>
> {
    val codes = mutableListOf<CodeMd>()
    var i = 0
    while (i < lines.size) {
        val t = lines[i].trim()
        if (t.startsWith("```")) {
            val lang = t.removePrefix("```").trim()
            val content = mutableListOf<String>()
            var j = i + 1
            while (j < lines.size && lines[j].trim() != "```")
            {
                content.add(lines[j])
                j++
            }
            if (j < lines.size) j++ // 跳过闭合 ```
            codes.add(CodeMd(lang, content.joinToString("\n"),
i, j))

            i = j

```

```

        } else {
            i++
        }
    }
    return codes
}

/** 代码块 → 独立 View (语言横幅 + 等宽代码 + 长按复制——dsh CodeBlock chrome)。
 */
/** 数学块视图：水平居中 + 数学位图 (j latexmath 本地渲染)；失败回退原文等宽显示。 */
private fun buildMathView(mb: MathMd): android.widget.LinearLayout
out {
    val box = android.widget.LinearLayout(this).apply {
        orientation = android.widget.LinearLayout.HORIZONTAL
        gravity = android.view.Gravity.CENTER
        layoutParams = android.widget.LinearLayout.LayoutParams
(
            android.widget.LinearLayout.LayoutParams.MATCH_PARENT,
            android.widget.LinearLayout.LayoutParams.WRAP_CONTENT,
        ).apply { topMargin = 4; bottomMargin = 4 }
    }
    val drawable = com.mov.android.math.MathRenderer.renderLate
x(
        mb.latex,
        18f * resources.displayMetrics.density,
        if (isDark) 0xFFE8EAE8.toInt() else 0xFF1F2329.toInt(),
        resources.displayMetrics.density,
    )
    if (drawable != null) {
        box.addView(android.widget.ImageView(this).apply {
            setImageDrawable(drawable)
        })
    } else {
        box.addView(android.widget.TextView(this).apply {
            text = "$$" + mb.latex + "$$"
            textSize = 13f
            setTextColor(if (isDark) 0xFF8A9099.toInt() else 0x
FF5C6470.toInt())
            typeface = android.graphics.Typeface.MONOSPACE
        })
    }
    return box
}

private fun buildCodeBlockView(cb: CodeMd): android.widget.Line
arLayout {
    val box = android.widget.LinearLayout(this).apply {
        orientation = android.widget.LinearLayout.VERTICAL

```

```

        layoutParams = android.widget.LinearLayout.LayoutParams
(
        android.widget.LinearLayout.LayoutParams.MATCH_PARE
NT,
        android.widget.LinearLayout.LayoutParams.WRAP_CONTE
NT,
).apply { topMargin = 4; bottomMargin = 4 }
background = android.graphics.drawable.GradientDrawable
().apply {
        cornerRadius = 8f
        setColor(if (isDark) 0xFF1B1F25.toInt() else 0xFFF2
F4F7.toInt())
        setStroke(1, if (isDark) 0xFF2A2F36.toInt() else 0x
FFE3E7EB.toInt())
    }
}
// 语言横幅行 (语言标签 + 复制按钮——dsh CodeBlock chrome:banner + copy c
ontrol)
val bannerRow = android.widget.LinearLayout(this).apply {
    orientation = android.widget.LinearLayout.HORIZONTAL
    gravity = android.view.Gravity.CENTER_VERTICAL
    setPadding(12, 4, 12, 0)
}
if (cb.lang.isNotBlank()) {
    bannerRow.addView(android.widget.TextView(this).apply {
        text = cb.lang
        textSize = 10.5f
        setPadding(4, 4, 4, 2)
        setTypeface(android.graphics.Typeface.MONOSPACE, an
droid.graphics.Typeface.BOLD)
        setTextColor(0xFF8E96A0.toInt())
    }, android.widget.LinearLayout.LayoutParams(0, android.
widget.LinearLayout.LayoutParams.WRAP_CONTENT, 1f))
} else {
    bannerRow.addView(android.view.View(this), android.widg
et.LinearLayout.LayoutParams(0, 1, 1f))
}
box.addView(bannerRow)
// 代码内容 (等宽 + 语法高亮 + 长按选择复制)
val code = android.widget.TextView(this).apply {
    textSize = 12.5f
    setPadding(16, 8, 16, 12)
    setTextIsSelectable(true) // 长按复制 (先 selectable——后 setT
ext 保留 spans)
    typeface = android.graphics.Typeface.MONOSPACE
    setTextColor(if (isDark) 0xFFC3C8D0.toInt() else 0xFF1F
2329.toInt())
    text = highlightCode(cb.content)
}
box.addView(code)

```

```

        return box
    }

    /** 代码语法高亮（轻量正则分词——Darkula 风格：关键字紫 / 字符串橙 / 注释绿灰 / 数字浅绿）。 */
    private fun highlightCode(code: String): android.text.Spannable
StringBuilder {
        val sb = android.text.SpannableStringBuilder(code)
        // 双配色：深色 Dark+ / 浅色 Light+ (VS Code 风格)
        val cType = if (isDark) 0xFF4EC9B0.toInt() else 0xFF267F99.
toInt()
        val cKw = if (isDark) 0xFF569CD6.toInt() else 0xFF0000FF.to
Int()
        val cNum = if (isDark) 0xFFDCDCAA.toInt() else 0xFF098658.t
oInt()
        val cStr = if (isDark) 0xFFE8A87C.toInt() else 0xFFA31515.t
oInt()
        val cCom = if (isDark) 0xFF7FB069.toInt() else 0xFF008000.t
oInt()

        fun color(pattern: Regex, color: Int) {
            for (m in pattern.findAll(code)) {
                sb.setSpan(
                    android.text.style.ForegroundColorSpan(color),
                    m.range.first, m.range.last + 1,
                    android.text.Spanned.SPAN_EXCLUSIVE_EXCLUSIVE,
                )
            }
        }

        // 顺序：关键字 / 数字先 → 字符串 / 注释后（覆盖其内的关键字——保证字符串整体一色）
        // 类型青色 (VS Code Dark+ #4EC9B0)
        color(
            Regex("\\b(String|Int|Long|Double|Float|Boolean|Byte|Sh
ort|Char|Unit|Any|Nothing|List|Map|Set|Array|Object|Record|Class|In
tArray|LongArray|DoubleArray|FloatArray|BooleanArray|ByteArray|Char
Array|ShortArray|Boolean|Integer|HashMap|ArrayList|Optional|Excepti
on|Throwable|Runnable|Thread|Companion)\\b"),
            cType,
        )

        color(
            Regex("\\b(fun|val|var|if|else|when|for|while|return|cl
ass|object|import|package|def|const|let|function|from|in|is|null|tr
ue|false|this|override|private|public|internal|data|sealed|enum|int
|long|string|void|new|extends|implements|async|await|try|catch|thro
w|throwable|suspend|companion|interface|abstract|open|init|construc
tor|super|break|continue|do|use|require|check|lambda|yield|with|as|
elif|except|finally|global|nonlocal|pass|raise|assert|export|defaul
t|typeof|instanceof|delete|switch|case|static|final|synchronized|vo
latile|transient|native|struct|union|typedef|namespace|template|typ
ename|goto|sizeof|extern|register|operator|mutable|explicit|friend|
virtual|override|sealed|inner|lateinit|by|get|set|field|property|va
lue|channel|flow|collect|map|filter|take|drop|reduce|fold|distinct|

```

```

sorted|groupBy|associate|zip|partition|chunked|windowed)\b"),
    cKw, // 关键字
)
color(
    Regex("0x[0-9a-fA-F]+|0b[01]+|\\d+(_?\\d)*((\\.\\d+)?([
eE][+-]?\\d+)?)?"),
    cNum,
) // 数字
color(Regex("\\\"[^\\"\\n]*\\\"|'[^'\\n]*'"), cStr) // 字符串
color(Regex("//[^\\"\\n]*"), cCom) // 行注释
color(Regex("/\\\"*\\[\\s\\S]*?\\\"/"), cCom) // 块注释
// diff 高亮: - 删除行红 / + 添加行绿 (diff 惯例——文字色 + 淡背景)
var offset = 0
for (line in code.split("\n")) {
    val start = offset
    val end = offset + line.length
    if (line.startsWith("-")) {
        sb.setSpan(
            android.text.style.ForegroundColorSpan(0xFFE05E
65.toInt()),
            start, end, android.text.Spanned.SPAN_EXCLUSIVE
_EXCLUSIVE,
        )
        sb.setSpan(
            android.text.style.BackgroundColorSpan(0x26E05E
65.toInt()),
            start, end, android.text.Spanned.SPAN_EXCLUSIVE
_EXCLUSIVE,
        )
    } else if (line.startsWith("+")) {
        sb.setSpan(
            android.text.style.ForegroundColorSpan(0xFF4FC3
7E.toInt()),
            start, end, android.text.Spanned.SPAN_EXCLUSIVE
_EXCLUSIVE,
        )
        sb.setSpan(
            android.text.style.BackgroundColorSpan(0x264FC3
7E.toInt()),
            start, end, android.text.Spanned.SPAN_EXCLUSIVE
_EXCLUSIVE,
        )
    }
    offset = end + 1 // 跳过 \n
}

return sb
}

/** 正则提取表格块: | 行 + 分隔行 (| : - - | : - - : | - - : |)。 */

```

```

private fun extractTables(lines: List<String>): List<TableMd> {
    val tables = mutableListOf<TableMd>()
    var i = 0
    while (i < lines.size) {
        if (lines[i].trimStart().startsWith("|") &&
            i + 1 < lines.size && MdTableParse.isSeparatorRow(l
ines[i + 1]))
        ) {
            val header = MdTableParse.parseRow(lines[i])
            val aligns = MdTableParse.parseAlign(lines[i + 1])
            val rows = mutableListOf<List<String>>()
            var j = i + 2
            while (j < lines.size && lines[j].trimStart().start
sWith("|")) {
                rows.add(MdTableParse.parseRow(lines[j]))
                j++
            }
            tables.add(TableMd(header, rows, aligns, i, j))
            i = j
        } else {
            i++
        }
    }
    return tables
}

```

```

/** 表格块 → HorizontalScrollView(dsh tableScroll)包裹 TableLayout。
 */
private fun buildTableViews(tb: TableMd, mw: io.noties.markwon.
Markwon): android.widget.TableLayout {
    val tl = android.widget.TableLayout(this).apply {
        layoutParams = android.widget.TableLayout.LayoutParams(
            android.widget.TableLayout.LayoutParams.WRAP_CONTEN
T,
            android.widget.TableLayout.LayoutParams.WRAP_CONTEN
T,
        )
        background = android.graphics.drawable.GradientDrawable
().apply {
            cornerRadius = 8f
            setColor(if (isDark) 0xFF1B1F25.toInt() else 0xFFF2
F4F7.toInt())
            setStroke(1, if (isDark) 0xFF2A2F36.toInt() else 0x
FFE3E7EB.toInt())
        }
        setPadding(12, 8, 12, 8)
    }
    // 表头(重线分隔 + seal 淡蓝块)
    tl.addView(buildRow(tb.header, tb.aligns, isHeader = true,
mw))
}

```

```

        tl.addView(bottomLine(2, if (isDark) 0xFF3A4256.toInt() else 0xFFE3E7EB.toInt()))
        // 数据行(轻线分隔)
        for (row in tb.rows) {
            tl.addView(buildRow(row, tb.aligns, isHeader = false, mw))
        }

        tl.addView(bottomLine(1, if (isDark) 0x262A2F36.toInt() else 0x1AE3E7EB.toInt()))
    }
    return tl
}

private fun buildRow(
    cells: List<String>, aligns: List<Int>, isHeader: Boolean,
    mw: io.noties.markwon.Markwon,
): android.widget.TableRow {
    val tr = android.widget.TableRow(this).apply {
        dividerDrawable = android.graphics.drawable.GradientDrawable().apply {
            setSize(1, 0)
            setColor(0x262A2F36.toInt())
        }
        showDividers = android.widget.LinearLayout.SHOW_DIVIDER_MIDDLE
    }
    val last = cells.size - 1
    for ((i, cellMd) in cells.withIndex()) {
        val align = aligns.getOrElse(i) { 0 }
        val tv = android.widget.TextView(this).apply {
            // dsh 表格专用字号(--dsw-font-markdown-table:略小于正文)
            textSize = 12.5f
            setTextColor(if (isDark) 0xFFE8EAEF.toInt() else 0xFF1F2329.toInt())
            if (isHeader) set Typeface(null, android.graphics.Typeface.BOLD)

            // dsh:首列 padding-left 0 / 末列 padding-right 0 (紧凑)
            setPadding(if (i == 0) 4 else 12, 10, if (i == last) 4 else 12, 10)
            minWidth = 100
            // 链接可点优先:不 selectable(selectable 与链接点击冲突——长按选择牺牲, 换取点击)

            setTextIsSelectable(false)
            movementMethod = android.text.method.LinkMovementMethod.getInstance()
            // 富内容:每格 Markwon 渲染(code/加粗/链接保留)+ 链接显式点击
            val cellSpanned = mw.toMarkdown(cellMd) as android.text.Spannable
            applyLinkClick(cellSpanned)
            text = cellSpanned
            // 列对齐(水平)+ 垂直居中(多行内容)

```

```

        gravity = android.view.Gravity.CENTER_VERTICAL or w
hen (align) {
    1 -> android.view.Gravity.CENTER_HORIZONTAL
    2 -> android.view.Gravity.RIGHT
    else -> android.view.Gravity.LEFT
}
// 表头色块 (seal 淡蓝底) ; 数据行透明
if (isHeader) setBackgroundColor(if (isDark) 0x265B
8DEF.toInt() else 0x1A2E6BE6.toInt())
}
tr.addView(tv)
}
return tr
}

/** 行分隔线 (表头重线 l3 / 数据行轻线 l2——对齐 dsh 双层级)。 */
private fun bottomLine(px: Int, color: Int): android.view.View
{
    val line = android.view.View(this)
    line.layoutParams = android.widget.TableLayout.LayoutParams
(
    android.widget.TableLayout.LayoutParams.MATCH_PARENT, p
x,
)
    line.setBackgroundColor(color)
    return line
}

private fun renderHistory(s: com.hermes.dsh.session.Session) {
    try {
        val blocks = s.deriveMessages()
        android.util.Log.i("MOV-Persist", "renderHistory blocks
= " + blocks.size + " events=" + s.events.size)
        if (markstreamMode && !markstreamBroken) {
            // 房间页: 整房历史一次灌入 (每房间 1 次 bundle 解析; onSettled 撤遮罩 +
滚底)

            val sb = StringBuilder("[")
            var first = true
            var count = 0
            for (msg in blocks) {
                val imgBlock = msg.content
                    .filterIsInstance<com.hermes.dsh.llm.Conten
tBlock.Image>()
                    .firstOrNull()
                var text = msg.content
                    .filterIsInstance<com.hermes.dsh.llm.Conten
tBlock.Text>()
                    .joinToString("") { it.text }
                // 带图消息: 照片文件在 → 气泡带缩略图; 文件没了 → [图片] 占位 (历史不裂)
                var imageUrl: String? = null

```

```

        if (imgBlock != null) {
            val f = java.io.File(imgBlock.attachment.id
)

            if (f.isFile) {
                imageUrl = fileToPageUrl(f)
            } else {
                text += " [图片]"
            }
        }
        if (text.isBlank()) continue
        val role = when (msg) {
            is com.hermes.dsh.llm.Message.UserMessage -
> "user"

            is com.hermes.dsh.llm.Message.AssistantMess
age -> "ai"

            else -> continue
        }
        if (!first) sb.append(',')
        first = false
        sb.append("{\"role\":\"" + role + "\"}").append(role).append("
\"," + "md\":\"").append(jsString(text))
        if (imageUrl != null) sb.append("," + "image\":\"").
append(jsString(imageUrl))
        sb.append('}')
        count++
    }
    sb.append(']')
    if (count > 0) {
        // 超时兜底：页面异常时 10s 强制撤遮罩
        android.os.Handler(android.os.Looper.getMainLoo
per()).postDelayed({ hideRoomLoading() }, 10000)
        getOrCreateRoomView().setHistory(sb.toString())
    } else {
        hideRoomLoading()
    }
    if (blocks.isNotEmpty()) {
        appendLog("—— 已恢复历史 (${s.events.size} 事件) ——")
    }
    scrollChatToBottom()
    return
}

// 原生降级路径 (markstream 不可用)：逐条原生渲染，全部无复制图标 (干净)
for (msg in blocks) {
    val text = msg.content
        .filterIsInstance<com.hermes.dsh.llm.ContentBlo
ck.Text>()

        .joinToString("") { it.text }
    if (text.isBlank()) continue
    when (msg) {
        is com.hermes.dsh.llm.Message.UserMessage -> ap

```

```

pendUserBubble(text)
        is com.hermes.dsh.llm.Message.AssistantMessage
-> appendAiMd(text, withIcon = false)
        else -> {}
    }
}
if (blocks.isNotEmpty()) {
    appendLog("—— 已恢复历史 (${s.events.size} 事件) ——")
}
hideRoomLoading()
scrollChatToBottom()
} catch (e: Exception) {
    android.util.Log.e("MOV-Persist", "renderHistory 失败: "
+ e.message, e)
    hideRoomLoading()
}
}

/**
 * 滚动聊天区到底部（最新消息）。历史 AI 回复的 markstream WebView 渲染完成后
 * 还会异步测高（fitWebViewHeight 最晚 ~1.8s），一次滚动会被后续 resize 顶偏，
 * 故立即 + 800ms + 2200ms 分三次兜底。
 */
private fun scrollChatToBottom() {
    val sv = chatScroll ?: return
    sv.post { sv.fullScroll(android.view.View.FOCUS_DOWN) }
    sv.postDelayed({ sv.fullScroll(android.view.View.FOCUS_DOWN)
}, 800)
    sv.postDelayed({ sv.fullScroll(android.view.View.FOCUS_DOWN)
}, 2200)
}

/** E4 权限门一键验证（点按钮自动跑全流程）。 */
/** E4 权限门一键验证（2 模式：默认权限 / 允许完全访问）。 */
/** 手动切换权限模式（default <-> open），按钮可见可点。 */
/** D4：审批策略写事件（B3 落盘——崩溃恢复自动恢复模式）。default=ask / open=never。 */
/

private fun recordApprovalPolicy(mode: com.hermes.dsh.tools.PermissionGuard.Mode) {
    try {
        val s = session ?: return
        val policy = if (mode == com.hermes.dsh.tools.PermissionGuard.Mode.DEFAULT_PERMISSION) "ask" else "never"
        s.append("approval/policy", mapOf("policy" to policy))
    } catch (_: Exception) {
    }
}

/** D4 启动恢复：折叠日志 approval/policy → 恢复模式（日志为唯一权威，内存只是投影）。 */
private fun restoreApprovalPolicy() {

```

```

        try {
            val s = session ?: return
            val g = permissionGuard ?: return
            for (index in s.events.indices.reversed()) {
                val ev = s.events[index]
                if (ev is com.hermes.dsh.session.SessionEvent.ApprovalPolicy) {
                    val isAsk = ev.policy == "ask"
                    g.setMode(
                        if (isAsk) com.hermes.dsh.tools.PermissionGuard.Mode.DEFAULT_PERMISSION
                        else com.hermes.dsh.tools.PermissionGuard.Mode.FULL_ACCESS,
                    )
                    modeBtn.text = "模式: " + (if (isAsk) "default" else "open")
                    appendLog("恢复审批策略: " + ev.policy)
                    return
                }
            }
        } catch (_: Exception) {
        }
    }

    /** D4 弹窗内容:工具名 + 参数摘要(敏感字段截断脱敏)。 */
    private fun buildApprovalMessage(info: com.hermes.dsh.tools.ApprovalService.PendingInfo): String {
        val nl = System.lineSeparator()
        val sb = StringBuilder()
        sb.append("AI 请求执行:").append(info.toolName).append(nl).append(nl)
        info.reason?.takeIf { it.isNotBlank() }?.let { sb.append("原因:").append(it).append(nl).append(nl) }
        val args = info.args
        if (args.isNotEmpty()) {
            sb.append("参数摘要:").append(nl)
            for ((k, v) in args.entries.take(6)) {
                var value = v?.toString() ?: ""
                // 脱敏:key/secret/password/token 字段截断
                if (k.contains("key", true) || k.contains("secret", true) || k.contains("password", true) || k.contains("token", true)) {
                    value = if (value.length > 8) value.take(4) + "*****" else "*****"
                } else if (value.length > 60) {
                    value = value.take(57) + "... "
                }
                sb.append(" ").append(k).append(" = ").append(value).append(nl)
            }
        }
    }

```

```

        if (args.size > 6) sb.append("    ... 共 ").append(args.size)
        .append(" 项参数").append(nl)
    }
    sb.append(nl).append("60 秒无响应将自动拒绝。")
    return sb.toString()
}

private fun togglePermissionMode() {
    val g = permissionGuard ?: return
    setPermissionMode(g.currentMode != com.hermes.dsh.tools.PermissionGuard.Mode.DEFAULT_PERMISSION)
}

/**
 * 审批模式统一入口（顶栏模式按钮 / 设置页共用；ask=true 每次确认，false 免确认）：
 * 与 permission.set_mode 同一条 guard.setMode + recordApprovalPolicy 路径——事件落盘，
 * restoreApprovalPolicy 重启可恢复。
 */
private fun setPermissionMode(ask: Boolean) {
    val g = permissionGuard ?: return
    val next = if (ask) com.hermes.dsh.tools.PermissionGuard.Mode.DEFAULT_PERMISSION
    else com.hermes.dsh.tools.PermissionGuard.Mode.FULL_ACCESS
    if (g.currentMode == next) return
    g.setMode(next)
    recordApprovalPolicy(next)
    val label = if (ask) "default" else "open"
    modeBtn.text = "模式：" + label
    appendLog("权限模式已切换 -> " + label + "（默认权限：只读放行+写类弹窗确认 / open：全放，高危子集仍确认）")
}

/** 请求是否明显需要执行工具（口头承诺兜底的后发式）。 */
private fun looksLikeToolRequest(text: String): Boolean {
    val t = text.lowercase()
    return listOf(
        "运行", "打开", "关闭", "设置", "执行", "查询", "查看", "调用", "测试",
        "open", "close", "set", "run", "turn", "execute", "call",
        "check", "get", "list",
    ).any { t.contains(it) }
}

/** 压缩日志落盘（filesDir/compaction.log，追加）——证据可查。 */
private fun compactionFileLogger(): (String) -> Unit = { msg ->
    try {
        val f = java.io.File(filesDir, "compaction.log")
        f.appendText(msg + "\n", Charsets.UTF_8)
    } catch (_: Exception) {
    }
}

```

```

    }

    /** E3:工具面模式标签。 */
    private fun modeLabel(): String = when (presentationMode) {
        com.hermes.dsh.tools.ToolPresentationMode.BOTH -> "both"
        com.hermes.dsh.tools.ToolPresentationMode.CODE -> "code"
        com.hermes.dsh.tools.ToolPresentationMode.NATIVE -> "native"
        com.hermes.dsh.tools.ToolPresentationMode.HARDWARE -> "hardware"
        com.hermes.dsh.tools.ToolPresentationMode.CAUSAL -> "causal"
    }

    /** 硬件类工具判定 (HARDWARE 模式的工具面白名单)。 */
    private fun isHardwareTool(name: String): Boolean =
        name.startsWith("torch.") ||
        name in setOf(
            "vibrate", "volume.get", "volume.set", "brightness.get", "brightness.set",
            "silent.on", "silent.off", "screen.on", "battery.status",
        )

    /** 浏览器 AI 能力是否已安装 (市场「浏览器自动化」内建包状态)。 */
    private fun browserAiEnabled(): Boolean =
        marketCapability?.builtinEnabled("browser-automation") == true

    /**
     * 浏览器 AI 工具同步 (安装 / 卸载 / 启停后调用) ——dsh 形态:
     * 安装 = BrowserMcpTools.connectProxy 挂载 (mcp__browser__ 入 agent / MCP 双面);
     * 卸载 = 按前缀 mcp__browser__ dispose (注册 / 注销对偶)。
     */
    private fun syncBrowserAiTools() {
        if (browserAiEnabled()) {
            val proxy = com.hermes.mov.browser.BrowserMcpTools.connectProxy(browserHandlers)
            if (proxy.isNotEmpty()) mountExtTools(proxy)
        } else {
            unmountExtTools(com.hermes.mov.browser.BrowserMcpTools.CLIENT_PREFIX)
        }
        android.util.Log.i(
            "MOV-Market",
            "浏览器 AI 能力 (mcp__browser__): " +
            (if (browserAiEnabled()) "已连接 (${com.hermes.mov.browser.BrowserMcpTools.proxyToolNames().size} 工具入面)"
            else "已断开 (工具面已收)")
        )
    }

```

```

    )
}

/** E3:按呈现模式过滤 agent 工具面 (both 全量 / code 只留执行器 / native 去执行器 /
hardware 只留硬件 / causal 只留因果图谱)。 */
private fun rebuildAgentTools() {
    val uiOnly = uiOnlyMcpTools
    val all = mcpHandlers.keys.filter { it !in uiOnly }
    val filtered = when (presentationMode) {
        com.hermes.dsh.tools.ToolPresentationMode.BOTH -> all
        com.hermes.dsh.tools.ToolPresentationMode.CODE -> all.f
filter { it in codeTools }
        com.hermes.dsh.tools.ToolPresentationMode.NATIVE -> all
.filter { it !in codeTools }
        com.hermes.dsh.tools.ToolPresentationMode.HARDWARE -> a
ll.filter { isHardwareTool(it) }
        com.hermes.dsh.tools.ToolPresentationMode.CAUSAL -> all
.filter { it.startsWith("causal.") }
    }
    agentToolSchemas = filtered.map { name ->
        com.hermes.dsh.llm.ToolSchema(
            name = name,
            description = "MOV 工具: $name",
            // 真实参数 schema:AI 需要知道参数才能正确调用 (空 schema 会导致盲调 / 参数
永远为空)

            parameters = toolParamSchemas[name]
                ?: mapOf("type" to "object", "properties" to em
ptyMap<String, Any?>()),
        )
    }

    // E3:执行层同步工具面白名单——不在当前模式的工具, agent 调用也 TOOL_NOT_FOUND (场缩
约束执行)

    agentToolScheduler?.allowedTools = filtered.toSet()
    android.util.Log.i("MOV-Boot", "工具面模式: " + modeLabel() + "
(agent 工具面 " + agentToolSchemas.size + " 工具)")
}

/** 通知按钮 (getActivity 中转): 点击 → 打开 App → 自动完成审批 (不弹窗) */
override fun onNewIntent(intent: Intent) {
    super.onNewIntent(intent)
    setIntent(intent)
    val action = intent.action
    if (action == com.hermes.dsh.tools.NotificationAnswerer.ACTION_ALLOW
ACTION_DENY
        || action == com.hermes.dsh.tools.NotificationAnswerer.
ACTION_DENY
    ) {
        val requestId = intent.getStringExtra(com.hermes.dsh.to
ols.NotificationAnswerer.EXTRA_REQUEST_ID) ?: return
        val allowed = action == com.hermes.dsh.tools.Notificati

```

```

onAnswerer.ACTION_ALLOW
        com.hermes.dsh.tools.NotificationAnswerer.completeFromB
roadcast(this, requestId, allowed)
        appendLog("[审批] 通知按钮: " + (if (allowed) "允许" else "拒绝"
) + "(" + requestId + ")")
    }
}

override fun onStart() {
    super.onStart()
    isVisible = true
    android.util.Log.i("ApprovalVis", "onStart visible=true")
    // 通知栏审批接管:回前台时,挂起的审批改为弹窗(通知 = 提示+入口,按钮由 MIUI 过滤)
    val pending = com.hermes.dsh.tools.NotificationAnswerer.takePending()
    if (pending != null) {
        val deferred = kotlinx.coroutines.CompletableDeferred<Boolean?>()
        val dialogRef = arrayOfNulls<android.app.AlertDialog>(1)

        runOnUiThread {
            val dialog = android.app.AlertDialog.Builder(this)
                .setTitle("审批请求")
                .setMessage(buildApprovalMessage(pending))
                .setPositiveButton("允许") { _, _ -> deferred.complete(true) }
                .setNegativeButton("拒绝") { _, _ -> deferred.complete(false) }
                .setOnCancelListener { deferred.complete(null) }

            .create()
            dialog.setCanceledOnTouchOutside(false)
            dialogRef[0] = dialog
            dialog.show()
        }
        kotlinx.coroutines.GlobalScope.launch {
            try {
                // 60s 超时:弹窗无人回应按拒绝处理(fail-closed,与通知栏审批超时一致)

                val result = kotlinx.coroutines.withTimeoutOrNull(60_000) { deferred.await() }
                com.hermes.dsh.tools.NotificationAnswerer.completeFromApp(this@MainActivity, pending.requestId, result)
            } finally {
                runOnUiThread { dialogRef[0]?.dismiss() }
            }
        }
    }
}

```

```

    override fun onStop() {
        super.onStop()
        isVisible = false
        android.util.Log.i("ApprovalVis", "onStop visible=false")
    }

    override fun onDestroy() {
        super.onDestroy()
        com.hermes.dsh.tools.NotificationAnswerer.unregister(this)
        // 侧边栏 WebView (抽屉面板内) 销毁 (不 destroy 泄漏 native 资源)
        try { sidebarWebView?.destroy() } catch (_: Exception) {}
        sidebarWebView = null
        // 本地 MCP 服务器 (:8389) ——不 stop 端口占用到进程死
        try { mcpServer?.stop() } catch (_: Exception) {}
        mcpServer = null
        // 在途协程 (启动恢复 / 切换房间 / 审批等待等) 全撤
        scope.cancel()
        // 所有房间页 WebView (含缓存房间里的) 统一销毁 (不 destroy 泄漏 native 资源)
        synchronized(liveMarkstreams) {
            for (mv in liveMarkstreams) {
                try { mv.view.destroy() } catch (_: Exception) {}
            }
            liveMarkstreams.clear()
        }
        roomViewCache.clear()
        // TTS 引擎解绑
        tts?.shutdown()
        tts = null
    }

    /** E3:循环切换工具面模式 (MCP presentation.set_mode 可调;UI 按钮已移除) both -
    > code -> native -> hardware -> causal -> both. */
    private fun togglePresentationMode() {
        presentationMode = when (presentationMode) {
            com.hermes.dsh.tools.ToolPresentationMode.BOTH -> com.hermes.dsh.tools.ToolPresentationMode.CODE
            com.hermes.dsh.tools.ToolPresentationMode.CODE -> com.hermes.dsh.tools.ToolPresentationMode.NATIVE
            com.hermes.dsh.tools.ToolPresentationMode.NATIVE -> com.hermes.dsh.tools.ToolPresentationMode.HARDWARE
            com.hermes.dsh.tools.ToolPresentationMode.HARDWARE -> com.hermes.dsh.tools.ToolPresentationMode.CAUSAL
            com.hermes.dsh.tools.ToolPresentationMode.CAUSAL -> com.hermes.dsh.tools.ToolPresentationMode.BOTH
        }
        rebuildAgentTools()
        appendLog("工具面模式已切换 -> " + modeLabel() + " (both 全量 / code 只留执行器 / native 去执行器 / hardware 只留硬件 / causal 只留因果图谱)")
    }
}

```